



GRAND CANYON
UNIVERSITY™

Pi-Piper

Peyton Hollenback

CST-451 Capstone Project Final Architecture & Design

Grand Canyon University

Instructor: Professor Mark Reha

Revision: 1.4

Date: 02/16/25

ABSTRACT

Republic Pi's current order management system involves manually comparing prices between Sysco and US Foods using a paper prep list and an iPad. To streamline this process, the proposed app will fully digitize and automate the restaurant's prep list and order management system. On the admin side of the app, it will compare prices between suppliers, generate the most cost-effective orders, and offer suggestions based on past data and current needs. Also, exclusively on the admin side will feature a prep list and ingredient CRUD tool. Additionally, the app will feature a visualized prep list and planning tool to assist employees in pre-open and pre-rush preparation. With presets and automated suggestions, the app reduces the need for manual input, optimizing inventory restocking and prep processes. The aim is to create a lightweight, dynamic admin system that saves time, increases productivity, and requires minimal oversight, ensuring that managers can focus on other critical aspects of running the restaurant.

History and Signoff Sheet

Change Record

Date	Author	Revision Notes
11/17	P. Hollenback	Revision One
02/16	P. Hollenback	Revision 1.4

Overall Instructor Feedback/Comments

Overall Instructor Feedback/Comments

Integrated Instructor Feedback into Project Documentation

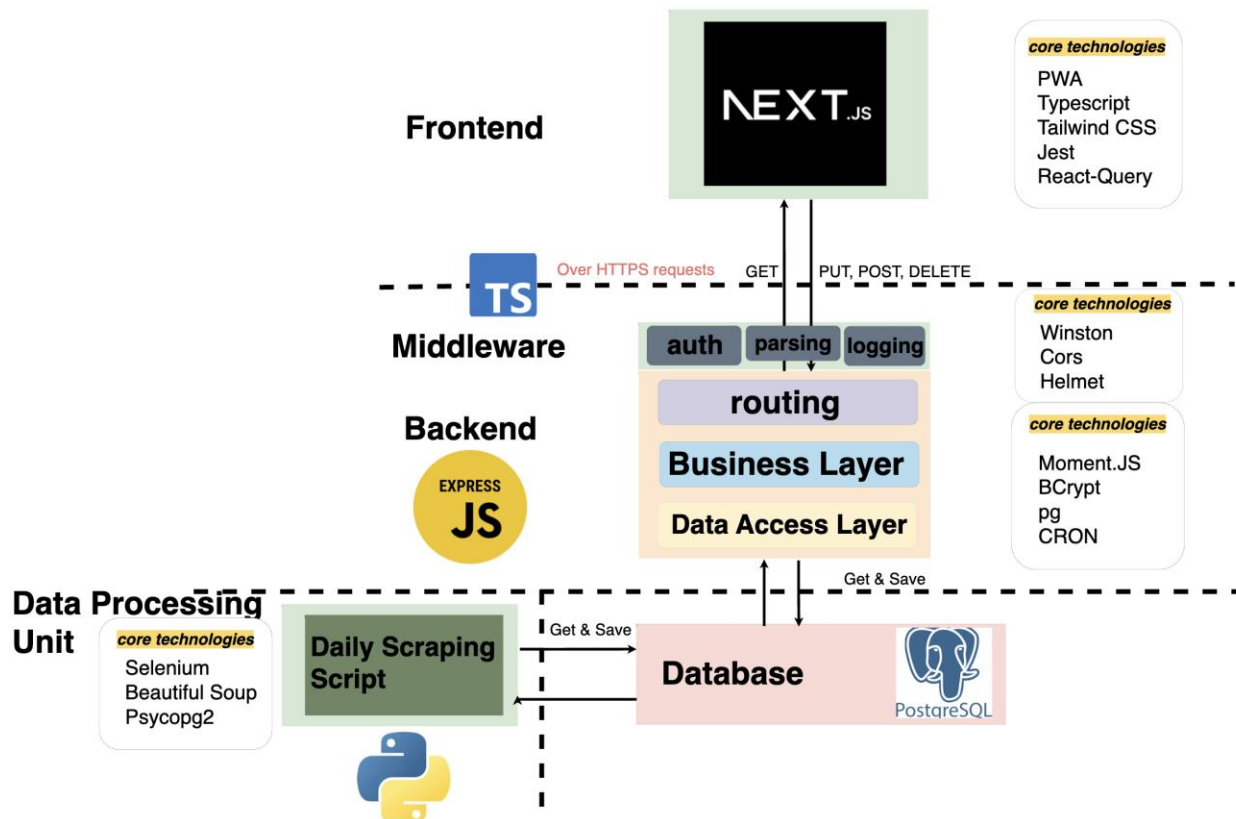
☐ Yes ☐ No

TABLE OF CONTENTS

DESIGN OVERVIEW	4
DETAILED HIGH-LEVEL SOLUTION DESIGN	5
DETAILED TECHNICAL DESIGN.....	12
APPENDIX A – TECHNICAL ISSUE AND RISK LOG	67
APPENDIX B – REFERENCES	8
APPENDIX C – EXTERNAL RESOURCES	69

Design Introduction

- Here is the high-level design that will be described in detail throughout this document

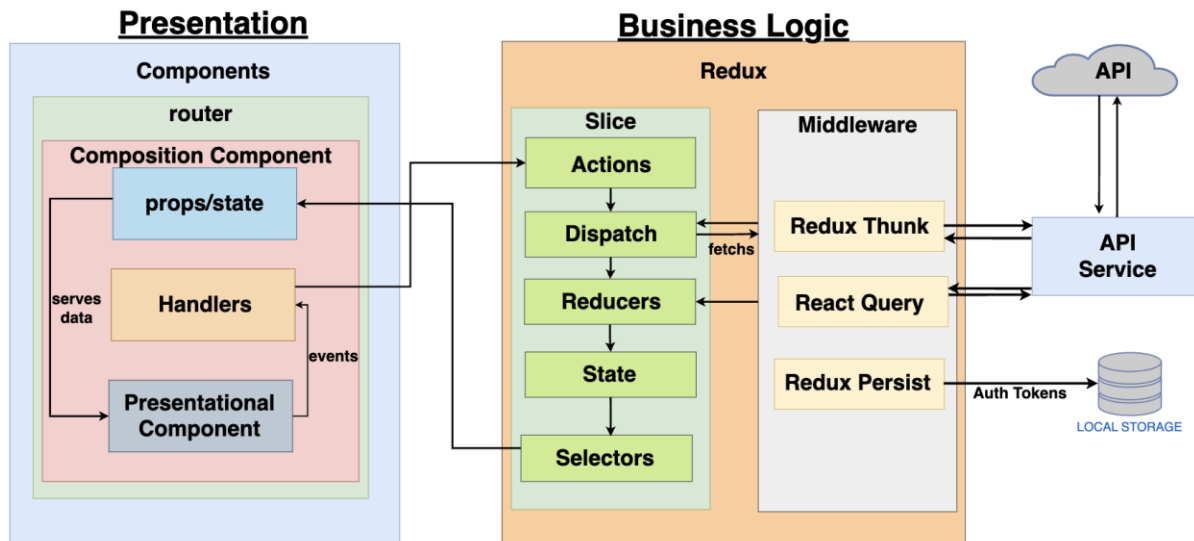


- Use the template to list the project deliverables that are to be included external to this Design Specification (Data Dictionary, API Design, etc.).

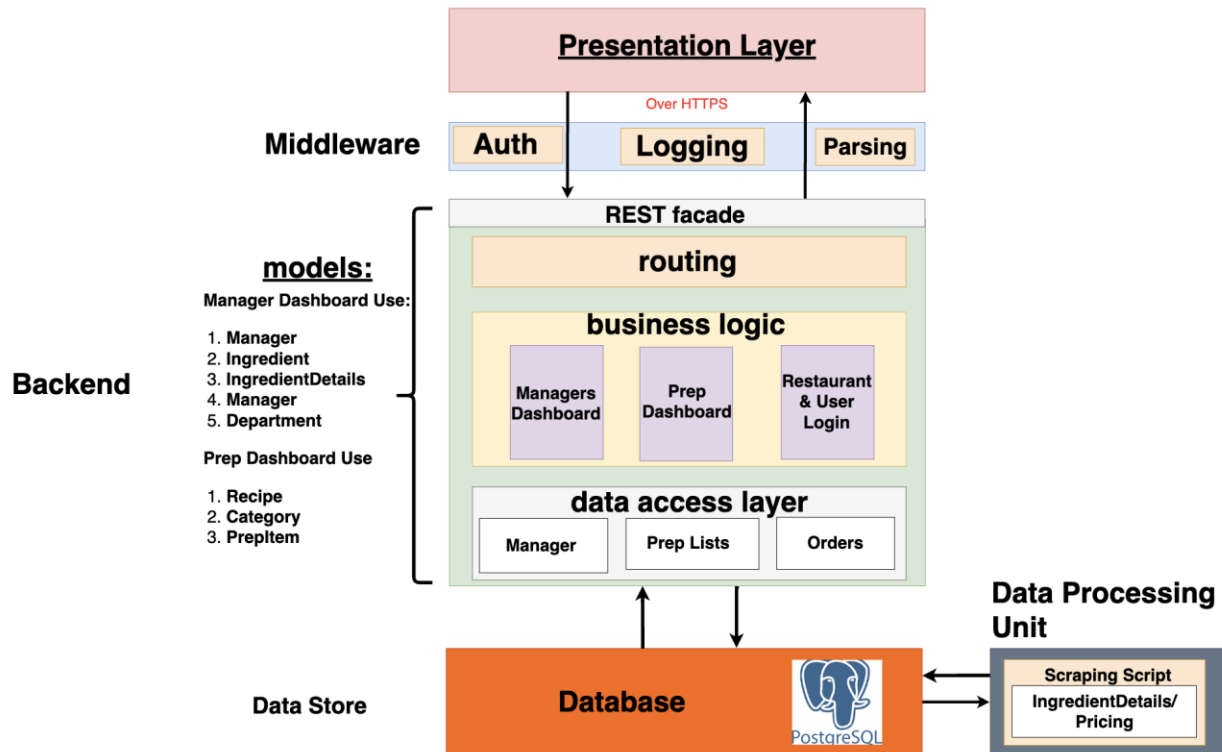
Deliverable Acceptance Log					
ID	Deliverable Description	Comments	Evaluator (internal or external as applicable)	Status	Date of Decision
1	Data Dictionary	A description of all tables and columns within the database.	External	Complete	11/16
2	ER Diagram	Entity Relation of the sole database of Pi Piper.	External	Complete	11/16
3	DDL Script	DDL script of said database.	External	Complete	11/16
4	API design	YAML file of API if the link is not working.	External	Complete	11/16
5					

Detailed High-Level Solution Design

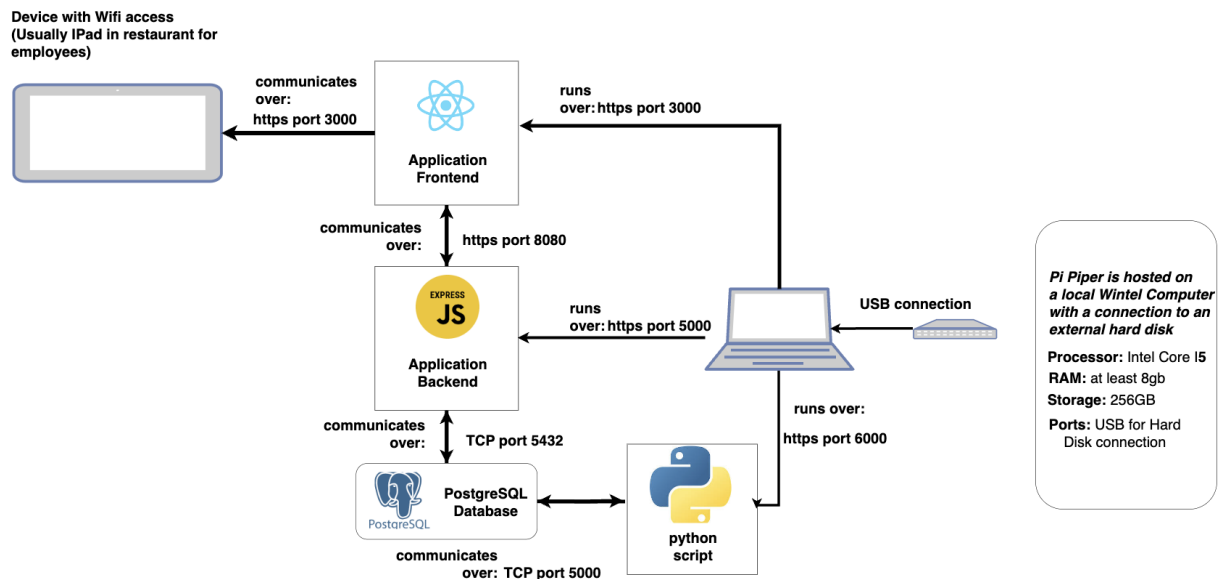
1. Provide a detailed overview of how the proposed design fits into the overall solution.
 - a. The following is a logical depiction of how the frontend will manage state and handle components as well as how the backend will serve data and implement business logic:



The frameworks used in this project will do a great deal of lifting to clean and post the data back to the backend. It is important that the state management using redux is clear and concise so there are no errors in modifying and saving data when altering the global state. Global state is an important aspect of Pi Piper as a whole, because the use of selectors and dispatching actions to perform upon state cleaned up each component immensely. Without the state management React offers the prop drilling would have created headaches for cleaning the data and posting them to the right endpoints consistently and error free.



b. The following is a physical depiction of how all elements within the system fit together using physical descriptions of connection like ports and networks:



The data backup I plan to implement as my key nonfunctional requirement will be done to ensure data integrity and well-maintained business insights. For me to complete this I need to get an external hard disk that will be physically attached to my hosting device (in this case a Wintel

computer). The following are the specifications that are required for both the hosting machine and hard disk:

Wintel:

Processor: Intel Core i5

RAM: at least 8gb

Storage: 256GB

Ports: an open USB port for connection with Hard disk drives

Hard Disk Drive:

Capacity: 1TB (considering long term future use and scalability)

Ports: USB connection port

This approach will require very little overhead on the part of the hosting machine so it can maintain its computing towards Pi Piper and will target the backup process to be done outside of store hours (between 1am and 4am). By following this key principle, introducing my NFR should be a noninvasive, mostly simple approach to ensuring that all transactions of restaurant operations and past entity entries and modifications will always be up to date. The same module which will signal the data processing script to commence, CRON will also handle the scheduling of data backups. This class will be robust enough to flag bad backups or incomplete daily data. Due to the use of the “Dim Date” table, the process of gathering all daily data from the data store and then posting to the frontend should be a simple process, but the parsing and testing of the validity of the data will be important once it is preparing for saving to the hard disk.

2. The following is a List of all Frameworks or third-party libraries that will be used.

Technology or Tool	Version	Description of Use
	FRONTEND	
React	19	Frontend framework that will handle the state of all pages, managing data before being posted to the backend.
Next	15.1.4	The way the react application is formatted and other routing and simplification tasks
React-dom	19.0.0	Helps manage the DOM.
React-Router	6.15	Handles navigation between different pages within the application.
React-Hook-Form	7.54.2	Helps manage forms across the application.
React-hot-toast	2.5.2	A UI popup that helps display success/failure messages

React-Icons	5.5.0	Icons.
MUI/Joy	5.5.0	Dashboard external styling library.
MUI/Material	6.4.1	Process external styling library
Zod	3.24.1	Helps manage forms and password validation.
Redux	8.2	Handles state management centrally for several screens within the application.
React-Query (Tanstack)	5.6	Manages state alongside fetching data from the backend.
TypeScript	5.4.4	Ensuring strong typing will make sure the data is always consistent.
Tableau	N/A	Creates data visualization graphs to improve BI reports.
Next-PWA	5.6.0	Allows a web application to double as a mobile application.
Tailwind CSS	3.4.1	Offers lots of custom CSS to keep the CSS minimal and speed up development with utility classes
Next-Auth	5.0.0	Handles user management, roles, authentication.
Zod	3.24.1	Checking return data from the backend source.
	MIDDLEWARE	
JWT	8.5.1	Secure authentication process for managers to gain access to the manager tools.
Winston	3.9.0	Logging library which allows multiple formats and outputs.
Body-Parser	1.20.1	Helps parse incoming request bodies, making it easier to access data sent from POST requests
Helmet	5.2.0	A collection of middleware functions that help secure Express applications.
CORS	2.8.52	Enables cross origin requests so that mobile and web

		clients can access the API all the same.
Express-Validator	2.20.33	Validates data types and formats error messages in return.
	BACKEND	
Node.JS	18.3	Writes JS on the server side of the application.
Express.JS	4.18	Web server framework used over the Node.JS language.
Drizzle Orm	0.40.0	Assists in DB mapping and typing.
dotenv	16.2	Environment variable management in Node.js.
BCrypt	5.12	For password hashing and encryption.
mysql2	3.12.0	Client which makes connections to a MySQL database.
cron	3.0.2	Task scheduling library to automate price updating.
	DATASOURCE	
PostgreSQL	14.9	Offers plenty of BI tools as well as visualization of time series data, which is helpful in this context.
	DATA PROCESSING UNIT	
Python	3.10.11	Used in the data processing unit.
Beautiful Soup	4.10.0	Parses HTML to extract data from websites
Selenium	4.0.0	Allows automation of sites to navigate a site and handle dynamically loaded content
Psycopg2	2.9	Allows connection to PostgreSQL database.
	TESTING	
React Testing Library	14.0.0	Tests React components.
Jest	29.13.2	For unit testing of JS and Typescript
Super test	6.6	Testing API endpoints for Express.
	DEV UTILITIES	

Swagger	N/A	For API documentation and design.
Babel	N/A	For JavaScript/TypeScript transpilation.

- a. The following table lists Proof of Concepts (POC) that were completed in an effort to ensure that the technologies and frameworks selected are the best fit for purpose, cost effective, and proper to solve the problem.

Proof of Concepts		
Description	Rationale	Results
1. Web Scraping using python in order to facilitate price comparison amongst two different companies. GOAL: confirm the viability of scraping Sysco and US Foods for price comparison in Pi Piper.	This proof of concept was essential for me to determine the viability of web scraping Sysco foods and Us Foods for Pi Piper price comparison. Although not an easy task, it was necessary I introduced myself to the concept of web scraping early to familiarize myself with the necessary technologies and the general flow that successful web scraping generally uses. I was unable to perform the proof of concept on the target websites of Sysco and Us Foods because they require a restaurant account to look through their sites. This scenario may call for more testing once I am authorized to use Republic Pi's restaurant account upon approval of the script's functionality that I created for this proof of concept.	For me to test some workarounds to errors when searching for items in an online marketplace such as ingredients, I scraped the websites for Instacart as well as Restaurant Depot. The script I wrote in python did not update a database, as that is a step too far at this stage and may require a separate testing stage so that I can test the results of loading data to a data source. The script I wrote simply outputs the price of two separate providers for a specific item and displays a null value if one provider did not have an item that the other did. This proof of concept overall went over extremely well and helped me understand the functions that both 'Beautiful Soup' and 'Selenium' offer. Another main benefit of developing this proof of concept was my understanding of initializing a python script and how it may be called within the larger Pi Piper application system.
2 - Redux state management to control which restaurant is logged in during a client's time using the application. GOAL: confirm that Redux can efficiently manage state globally, ensuring consistent and persistent data across Piper's components to avoid data reset issues and maintain operational efficiency.	Redux was one of my blind spots coming into the design of this application. But this proof of concept ensured I spent adequate time researching and implementing redux to a smaller scale application, while simultaneously deciding the design patterns within redux that would work best for this application. A key goal of this POC was to confirm that Redux could manage the "active restaurant" globally. This setup is essential for Pi Piper, as it enables the app to populate the various components with data specific to the signed-in restaurant, ensuring consistency in tasks, prep lists, and inventory data across the app. The ability to avoid data reset issues is paramount for the future of this application, if there is ever concern amongst management and employees about the state of restaurant operations, the use of this application will raise business concerns that can shatter the overall efficiency of the restaurant.	The key takeaways from this proof of concept are as follows: I was able to confirm the consistency in state that redux provides eloquently. I validated the design patterns concerning slices of state to maintain state management as modular as possible, ensuring the future scalability of the project. I revealed the numerous benefits redux provides concerning testability by researching and implementing Redux Dev Tools to log and test each dispatched action against state. These dev tools will absolutely go into Pi Piper mainly due to the benefits that snapshots of state provide to the debugging process. Overall redux has proven to be an essential aspect of the Pi Piper development process because of the need for data synchronization of core functionality like prep list updates, restaurant specific tasks/insights, and cart synchronization.
3 - Using PWA to test offline access to critical data. GOAL: To validate Pi Piper's ability to provide reliable offline access and enhanced usability through the PWA functionality, ensuring	Utilizing Progressive Web App (PWA) functionality can greatly improve the usability and limit the risk involved to a business including this application into their restaurant's operations. PWA applications enable offline access to critical data each time the application is	Upon the same application used to test how redux can improve the application, I implemented PWA functionality to check if it's possible for a React application to operate smoothly in low-connectivity environments.

consistent service for restaurant managers and employees even in low connectivity situations.	accessed. This POC can ensure the reliability of the Pi Piper application if say the Wi-Fi connection is down or the connection is unstable. Especially since the main target client of Pi Piper is the company iPad, the connections reliance comes into question. PWA has less effect on the system if a user is accessing the application with their phone, but PWA will assist in this regard also by allowing the website to be pinned to the user's home screen, as if it is a native app on any system. Overall, Pi Piper is almost a perfect use case for PWA and if done well can create a far more consistent and reliable service to restaurant managers and employees; while simultaneously creating a more intuitive application across nearly any client attempting to use the application.	By checking this with frequently accessed resources, like daily prep lists or critical inventory items I implemented the Service Worker to use the 'Network First' strategy. By having the service worker first check the server for the resources before checking the cache, the applications state should always be updated whenever possible, but when not possible it will grab the data from the cache that was stored upon the last successful request from the server. This proved successful, and the state was managed locally within the application without any posting to the backend. But the operations would not be interrupted. This POC, however, did reveal that there is really no workaround to unfortunate timing of losing network connection. If a network connection were to go out for a day starting before the new prep list is fetched for the next day, it would update from the previous (already completed) prep list. This introduced a new technical requirement of caching the submitted prep list for the next day as soon as the submission is confirmed. This would allow no loss of updated data for the most important section of the application which drives most all other functionality, the prep list completion. Also, this POC helped reveal that the signal that runs the script to update the pricing of ingredients can be improved by setting times to complete this if a request is unsuccessful the first time due to network connectivity. Overall, I was able to confirm the improvements that PWA offers to Pi Piper, while revealing some technical requirements to further improve the uptime of the application, even with network connectivity issues. If done correctly there may be little that can get in the way of some of the core components of Pi Piper, with enough foresight and planning of error handling in lots of different cases.
4 - JWT authentication of admins	Another blind spot of mine is with user authentication in React/Express. Because of authentication's level of importance in this system, I felt it was necessary that there be a POC dedicated to this concept. All admins must be authenticated before viewing the manager dashboard in Pi Piper, so not all endpoints will share the same level of protection; but especially when dealing with the business insights of a specific restaurant or managing carts on behalf of a restaurant's operations separation of users and security of a good portion of endpoints is extremely important.	Within the concept of Json Web Tokens, token expiration and refresh were the hardest aspect to implement. Pi Piper will utilize token expiration in the name of security because a lot of clients are reused in house, so it is good to refresh to keep some functionality out of the wrong hands. A key to this is with the radio button 'keep me signed in' which I tested with for this POC, and upon checking or unchecking this radio button upon login, the user logging in was able to maintain session state across sessions when they would like, and perform automatic logout when this radio button goes unchecked. Overall, the login persistence of admins and general role management offered by Pi Piper will provide both secure user management and a foundation for further user roles and logging in the future.

Hardware and Software Technologies	
1 – React – Frontend framework for handling the DOM, fetching data, and managing state.	
2 – Express.JS - The web server framework used for creating the backend of the application.	

3 – Python – Will be used strictly for web scraping functionality.
4 – PostgreSQL – The version of MySQL database this application will be using.
3 - Hard Disk Drive – will hold data backups in an external drive.
4 - Wintel Computer – For running the application locally at the restaurant.

Detailed Technical Design

General Technical Approach:

For explanation of the Key technical decisions, look out for notations(^x) and there provided explanations below in the ‘Key Technical Design Decisions’ tab.

Pi Piper will be developed using the React frontend framework to dynamically update or modify the DOM as well as manage the state of all data in the frontend before posting updated information to the backend to be saved to the datastore or complete some of the further business logic. Pi Piper requires global state for numerous components of the application; and when managing routing to different screens using React-Router these screens will be able to share all relevant state while eliminating prop drilling wherever possible. But not all screens will leverage redux. When there is data that is rarely mutated and tightly bound to the server state, this is where I will use the key technology React-Query to fetch and manage this state (^1). This likely will involve things such as restaurants, categories, managers, and so on. The key reason I ended up choosing React over some of the other front-end frameworks is due to the robustness of global state management React offers like Redux (^2). Redux will be relied upon to handle the state of data that is frequently modified by the user. This includes data like prep lists, cart management, and so on. Upon implementing ‘Pi Piper’ at the Republic Pi restaurant or one of its sister restaurants, it is crucial that the data being collected and served is entirely trustworthy. Any situation where data is corrupted- whether due to faulty business logic or improper loading into the wrong tables in the datastore- could introduce significant business risks, particularly in critical areas like prep lists and ordering functionality. Given the centrality of data integrity to Pi Piper’s core operations, ensuring reliable data management throughout the process is essential. As the developer I must have full confidence in the systems that facilitate the Extract, Clean and Load (ECL) processes, ensuring that all data is processed and served accurately before being posted to the backend or used to execute business logic. TypeScript will be utilized throughout the application to ensure the high level of integrity as previously described. This will provide several key benefits. First, it will enhance the testability of the application by utilizing libraries like ‘Jest’, which can validate correct typing across the entire application. Second, TypeScript enforces consistency and formality, so that the data from the frontend all the way through backend and datastore never loses form, ensuring that no part of the application handles data without a defined data structure. All the effort and technology invested in ensuring the application’s state integrity will be futile if a network connection issue results in corrupted data retrieval or failed postings to the web server. Because of this concern, React’s integration with PWA gave me all the information I needed when deciding on the framework to work with. PWA, or ‘Progressive Web App’ offers caching and loading of the critical data points even with limited network connectivity (^3), eliminating some of the risks involved in integrating an application paramount to the restaurant's operations. For consistency in design across the front end, Tailwind CSS will be leveraged to eliminate countless lines of custom CSS that would otherwise be necessary for every screen (^4). All in all, the front end of Pi Piper was designed to handle extensive data processing and presentation tasks, ensuring the business insights are reliable and consistent over time. The application must remain highly responsive and adaptable, minimizing downtime and maintaining functionality even in environments with limited or unreliable network connectivity.

Middleware is a thin aspect of the system overall but is important in one more way than one. Starting with intercepting HTTP requests to check whether a user is authenticated before proceeding with the processing of a request using Json Web Tokens. If an attempt is made to access to access a protected

endpoint without proper authorization, the middleware will immediately respond with an unsuccessful status code (such as 401 Unauthorized). After the authentication check, the middleware can also parse incoming request data, such as JSON using the Body-Parser library and attach the parsed data to the request object so that the backend can more easily access the data. Additionally, logging will take place within the middleware upon unauthorized or invalid access attempts or will log the information about successful access to endpoints, including user identification and resource accessed by utilizing the logging library Winston. As an extra precaution within the middleware, helmet will be leveraged to modify the HTTP headers during the request-response cycle. (^5) CORS is another necessary and easily injectable resource that can be leveraged within the middleware. It will easily disallow different types of clients to access the web server and create simple configurations to improve the protection of the backend in a straightforward manner. (^6)

Once the request has been authorized, parsed, and logged, the backend can begin its processes. Express is used to create a server that exposes endpoints to the HTTPS protocol. All endpoints create a RESTful interface between the frontend and the datastore, ensuring a seamless data flow and interaction. Each module within the backend relates to one entity in the datastore. In most cases, the following is used to facilitate the flow of data through the backend: routing, controller, services, and finally a data access layer. For some of the protected information, mostly inside of the manager module, the library BCrypt is used to hash and encrypt all passwords and emails so that there is no overexposure of this data along the data flow. The endpoint access and related classes contribute to the design approach that I try to maintain across the application as much as I can: modular and keeping aspects interrelated. This was generally easy within the backend because of its scope of service which is mostly serving data straight from the data store to the frontend or updating and deleting data from the store. But there are a few areas where the complexity of the store made this difficult, namely the prep completion and inventory transactions. For each prep list transaction, there will always be numerous subsequent inventory transactions for the ingredients that are related to one prep item(^Inv). The inventory is the only module within the backend that does not expose endpoints to alter the entity in the data store. There is a good amount of data transactions that need to take place with time stamp data fields that need to be stored. To perform this accurately, this will be handled within the services using the Moment.JS library to gather and structure current time variables. Although not crucial, the time of completion of prep items or time in between certain day to day operations of the restaurant can be important to future business insights if new insight screens are added outside of the inventory turnover page. There are a few modules that do not require endpoints to modify entities, only get them, such as restaurants and ingredient details. This is intentional, due to the fact that adding a new restaurant would require a whole slew of new features to service this new restaurant, which is not the kind of infrastructure that was ever intended for this application. In terms of the ingredient details, they should only be served and not allow modifications because these details are all modified by the data processing unit that handles the web scraping approach for the providers, which is entirely separate from the front and backend of Pi Piper and directly accesses and updates the data store. For these pricing updates to take place in a timely manner, it will rely on the backend to handle the scheduling tasks to decide when to run the data processing script. This can be accomplished by utilizing the CRON library to run the script whenever there is an adequate network connection. It is important that the script itself only has one task, since it holds some of the more complex functionality of the entire application. For more details about the data processing unit, view its description below.

The datastore is completely held within PostgreSQL. PostgreSQL holds many advantages when it comes to driving an application with the potential for lots of business insights. It visualizes data fluctuation over time which can be presented to stakeholders without extra coding involved. This is especially important for price fluctuation due to the daily storing of pricing for two providers, which calls for plenty of price

comparison. My database follows the data warehouse design scheme due to the need for transactional data like prep completion or daily pricing (^Pg).

The data processing unit will update pricing of ingredients once a day, preferably before employees come in in the morning at 6am. It is my intention that the script only maintains the one task to update pricing for a number of reasons. First, it is important because there is risk in passing around the username and password for Republic Pi to sign into Sysco or US Foods which could raise serious concerns for stakeholders. Second, it is generally good practice for scripts of this complexity to have little insight into how the rest of the application works; as this can only muddy the waters of development. The only interaction that this unit should receive from the backend is the CRON timed signal of where to start and nothing else. All necessary information the script needs is the ingredients for all restaurants that it needs to search for, which it can grab from the database, and it should be uploading its results back to the database. For the web scraping to be completed, the library Selenium will be used to navigate the site and perform logins when necessary (^7). And will work with beautiful soup to gather and parse the HTML to gather pricing information according to ingredient names. Both these libraries are crucial to the development of the script and neither could complete the job without the other.

Key Technical Design Decisions

[Any final technical design decisions, such as framework decisions, etc., should be documented here. This should list the technology/framework, its purpose in the design, and why it was chosen. If necessary, the proper Proof of Concepts should be defined and implemented to validate the technical decision.](#)

^1.

The fashion in which I implement the use of React-Query should improve the performance of Pi Piper for several reasons. First, React-Query simplifies synchronization with key configuration settings like 're-fetch on reconnect'. This setting automatically handles lots of syncing of data without having to write boiler plate code which would significantly bloat the codebase. Like a lot of the other key design decisions within Pi Piper, React-Query reduces the risk that comes with implementing this application into a restaurant with at times poor network connectivity. React-Query also maintains an edge over some of the other common HTTP libraries like Axios by intelligently caching frequently requested data to limit the number of network requests and overall improving the performance of the application. Outside of the performance of fetching data, React-Query also has plenty of built in features that can significantly improve the user experience: such as its built-in error and loading states which enables the use of loading/spinner icons(view the elements components for how this is implemented) to fill the space whenever the user is waiting for data to be served, eliminating "dead screens". Overall, a mixture of React-Query and Redux will be the keys to creating a highly performant application that has the ability to be relied upon heavily.

^2.

State management is handled by a few key technologies. The most important of them all is 'React-Redux'. Redux is what helps manage global state across Pi Piper. This technology is only utilized in a few pages within Pi Piper to limit prop drilling that can in some cases span across three or more levels of children, which can sometimes have to go through containers/components that have no relation to the data which is poor practice when there are tools like global state available. Redux also integrates nicely with next actions, which allows you to perform predefined changes to an application data's state, you can use selectors in any component within the project, and leverage middleware to handle asynchronous operations like API calls in a predictable manner. This creates a complete state management solution where actions trigger state changes, reducers process those changes, and components efficiently access only the data they need through selectors. In the future, it's good to remember that redux state management is not lightweight and should be used sparingly. In this case I think there is an argument to be made it could have been overkill even though it was used multiple times across the application.

Rshould not be utilized at all times across this app. Although it is a single source of truth, it creates concerns when saving back to data stores.

^3.

The worker that is implemented into the index file of the application will handle all caching of developer defined data points (like prep lists or ingredients) upon the user first loading up the app on a device. Each time the worker loads up or re-renders the application with a working network connection it will save that state to be served if the connection is faulty upon the next use of the application. PWA provides a quick patch to a major issue in this regard and will improve the application uptime and eliminate as many error screens as possible, which will improve the performance of this application tremendously. But this is not all that PWA offers. It is also essential in allowing different types of client access to the application. Whether it be an employee updating the prep list from their phone, a manager creating a prep plan for the next day, or an owner managing the prep items from their desktop, the experience will all be the same. The application will be able to be saved to the user's home screen on mobile devices for quick access to the application and will be loaded as if it is a native application, no matter what the operating system. PWA allows the application to be loaded without any internet search bars or otherwise functionality that would be loaded in a typical web browser application. The ability for the application to feel native to nearly any operating system without having to deal with the configuration of catering code to each OS is invaluable when trying to deliver a positive user experience in a timely fashion.

^4.

Tailwind nearly eliminates the need for any CSS files, effectively cleaning up the general solution design. Also, within the configuration file of Tailwind I can implement a consistent color palette and spacing across the application, which is extremely important to creating a frontend that looks sleek and intuitive. On top of this, Tailwind does the heavy lifting when it comes to a responsive design by providing responsive utilities that work well on various devices.

^5.

Helmet is a part of the middleware tier because of its functionality in modifying headers without extra configuration from the developer. One key cyber-attack method it provides is X-XSS-Protection. Upon the response being returned to the client, Helmet will add the necessary headers that will tell the browser to block reflected XSS attacks, so the browser will block any malicious or suspicious scripts that may be injected into Pi-Piper. This is one of the many attacks that helmet helps protect against; there is virtually no downside to including helmet as it can be injected with one line of code. It is important that tools such as helmets and other related libraries are preemptively included in Pi-Piper so that attacks can be intercepted without countless lines of code that would be necessary to account for these attacks and navigate them safely.

^6.

Like Helmet, CORS can be injected in one line, and the configurations that will address a lot of the safety concerns can be done in a few lines. By configuring the hosting domain as a trusted origin, it will only allow requests stemming from the frontend of the application to be allowed. There is little reason to add different trusted points of origin for this application, unless in the future there is an expansion of different frontends that need to use this backend.

^7.

Selenium offers functionality outside of just pressing buttons and searching ingredients as well. It is a key aspect of creating a robust script to ensure the attempted result almost always happens. It can help achieve

this by gracefully handling pagination and dynamically loading content to ensure the active HTML on the screen is always being given to beautiful soup for parsing and checking the values of the ingredient cards.

^Inv.

The following is pseudocode that further demonstrates how inventory transactions will be logged:

- I. Frontend sends request to backend with the following details
 - a. Prep Item Id
 - b. Quantity Completed
 - c. Restaurant Id
 - d. Time stamp of completion
- II. Backend receives request and routes the request to the proper resource, in this case to the controller method, 'Create Prep Completion'.
- III. The controller continues the flow to the service layer where the related ingredients are then fetched from the data store.
- IV. The service layer completes the calculations (multiplying ingredient quantities of one unit by quantity of what was prepped)
- V. After calculating the quantity of each ingredient used, the backend will create a inventory transaction to track the usage of ingredients

^Pg.

PostgreSQL is known for its ability to maintain a 'data warehouse' design scheme. In a data warehouse, tables are organized as either "Fact" or "Dimension" tables, providing a clear structure for logging transactions and maintaining business entities. Dimension tables store descriptive details related to business entities, such as ingredients, categories, or manager information, providing essential context for fact tables. Dimension tables are designed with more columns and fewer rows, which consolidates descriptive data and prevents redundancy. Importantly, dimension tables also serve as regular entities, accessible for Create, Read, Update, and Delete (CRUD) operations through the API, ensuring they can be maintained and updated as part of standard business processes. These tables capture quantitative data for analysis, representing core business metrics like inventory levels or prep task completions. With fewer columns and more rows, fact tables store measurable events linked to dimension tables, creating a compact, query-efficient structure ideal for reporting and analysis. This data warehouse design keeps fact tables streamlined for performance, while dimension tables provide the necessary context and can be managed directly via the API for flexibility and efficiency.

Database ER Diagram:

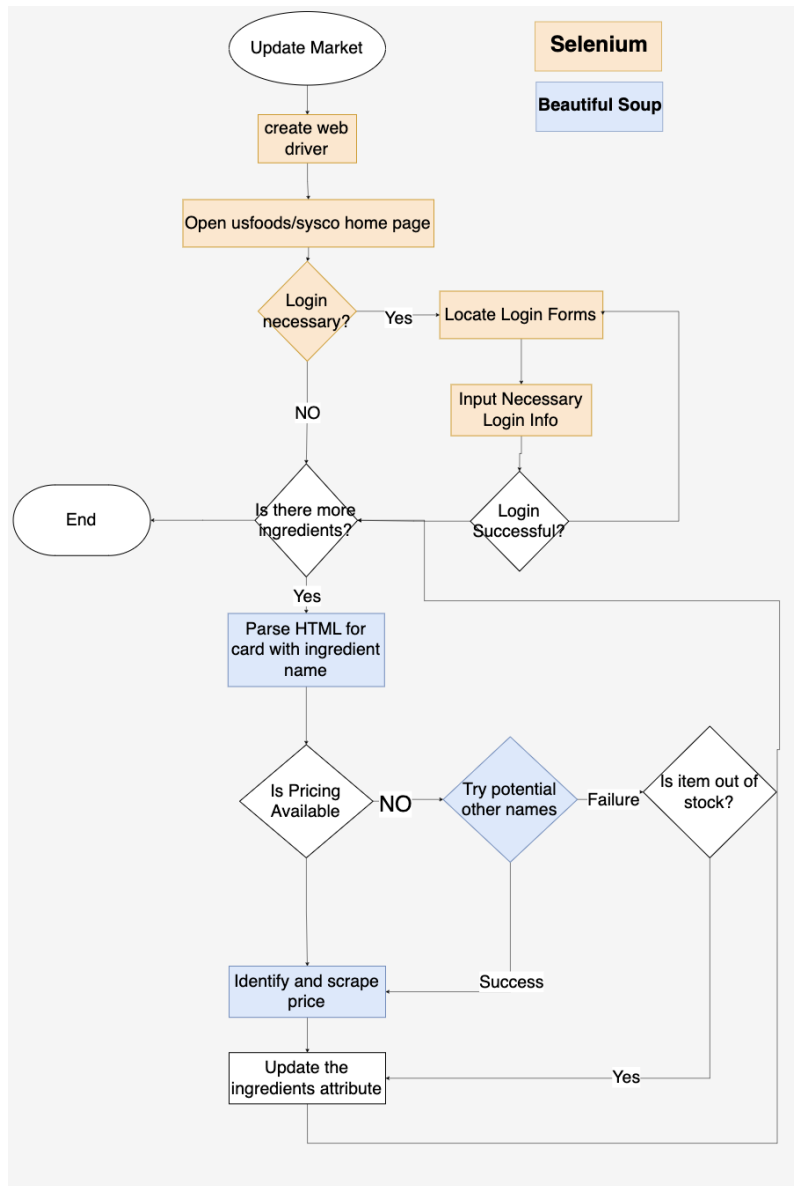
View the attached ER diagram to view how the database is set up.

Database DDL Scripts:

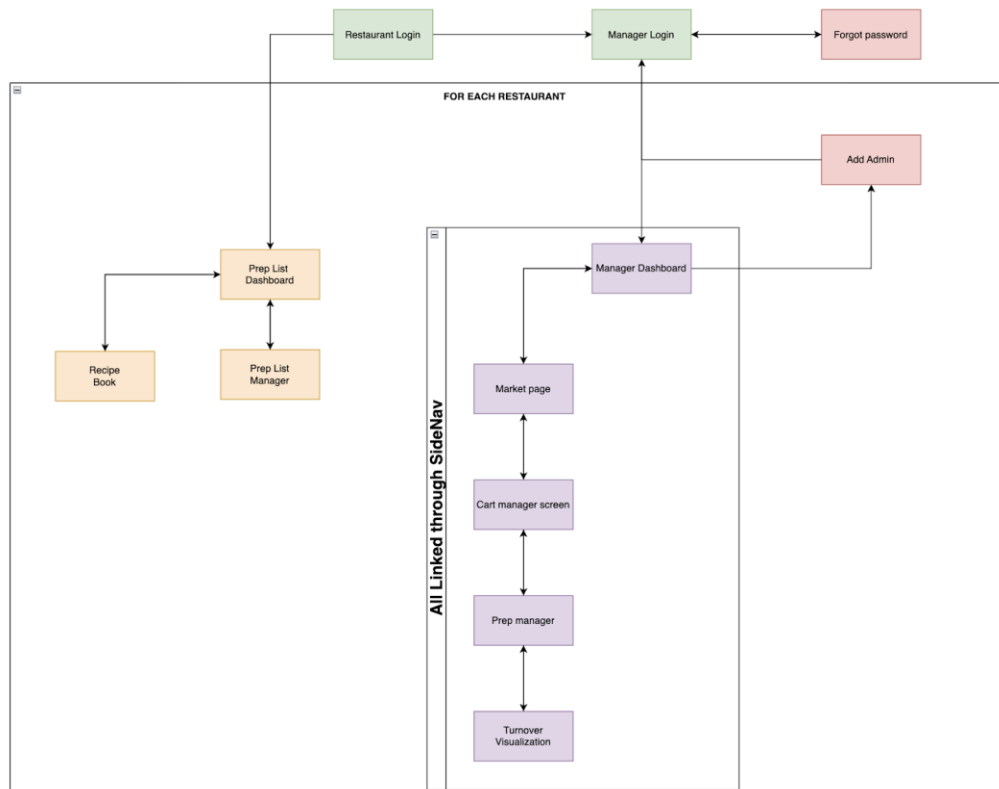
View the attached DDL script to view constraints and FK relations.

Flow Charts/Process Flows:

The following is a flowchart that maps out the functions of Selenium and BeautifulSoup in the scraping process.



Sitemap Diagram:



User Interface Diagrams:

The following is a breakdown of the component hierarchy of all composition components.

There will be an overview of how the components are laid out as well as a description of each child component of said composition, which will outline the necessary props they require.

Keep in mind a few key terms that help describe and categorize the components better.

Elements:

Before outlining the details within the components, first I will list the ‘element’ components that will be reused across the application. When looking through these reusable elements note the colors signifying how they will be outlined in the larger application. They will also define the props necessary to customize these components in any given scenario, when these elements are referenced within other components there will be the props for that specific instance of the element so that it fits the style and functionality specific to the screen.

Button:

props -->

label : What the button says

onClick : the method to handle the click

type : button type (submit, button, reset)
defaults to button

size : small, medium, or large button



Input:

props -->

placeholder : what the box says before typing begins

type : type of input like text or date

value : represents the current value of the input

onChange : handling the typing in the box

errorMessage : displays an error message below the input
when the validation fails



<<error message>>

Carousel:

props -->

items(array): contains data for each button

buttonWidth : width of buttons

buttonHeight : height of buttons

onButtonClick : the function that handles logic for a click

visibleButtons : the number of buttons that should be visible within the viewport at one time



Spinner/Loader:

props -->

color : the color of the spinner

message : what will be displayed next to the spinner

isLoading : specifies if the Loader should be displayed

size : signifies the size of the spinner



Radio Button:

props >

label : what text will be displayed next to the radio

onChange : the method that handles a change event

checked : determines if the radio button begins checked

size : determines how big the text and radio button are



Number Select:

props -->

value : the current value of the number input

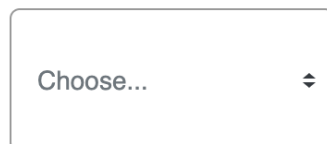
onChange : a function to handle changes in the selected number

min : the minimum number that can be selected

max : the maximum allowed value for the number input

step : the step value for incrementing or decrementing

label : what will display in the field



Dropdown Menu:

props -->

options : array of options to be displayed

placeholder : the text at the top of the dropdown

selectedValue : which option, if any, is currently selected

onSelect : method that handles the change of selection



Open this select menu

- One
- Two
- Three

Modal:

props -->

isOpen : controls the visibility of the modal

title : the heading message

footerActions : contains the options on the bottom
can be buttons or messages

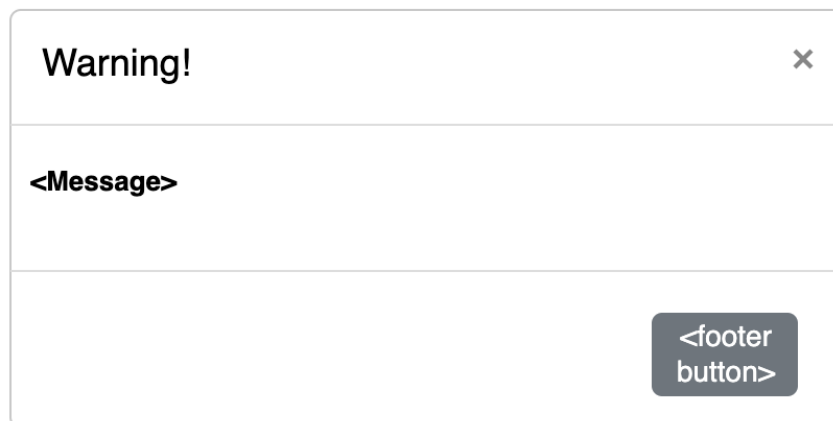
size : size of the modal

message : Message to display

isOpen : method to call if open

onClose : method to handle close

closeButton : shows a close button (x) in the top right of
the modal



*****To see the behaviors of each component and how they behave in relation to the state, View the connected UML in the 'UML' section directly below this section. Each component directly corresponds to a UML more thoroughly defined below. *****

RestaurantLogin:

Upon opening the app for the first time, the user will be navigated to the RestaurantLogin container component. This component holds the “Restaurant Decal” and “Initial Login” components.

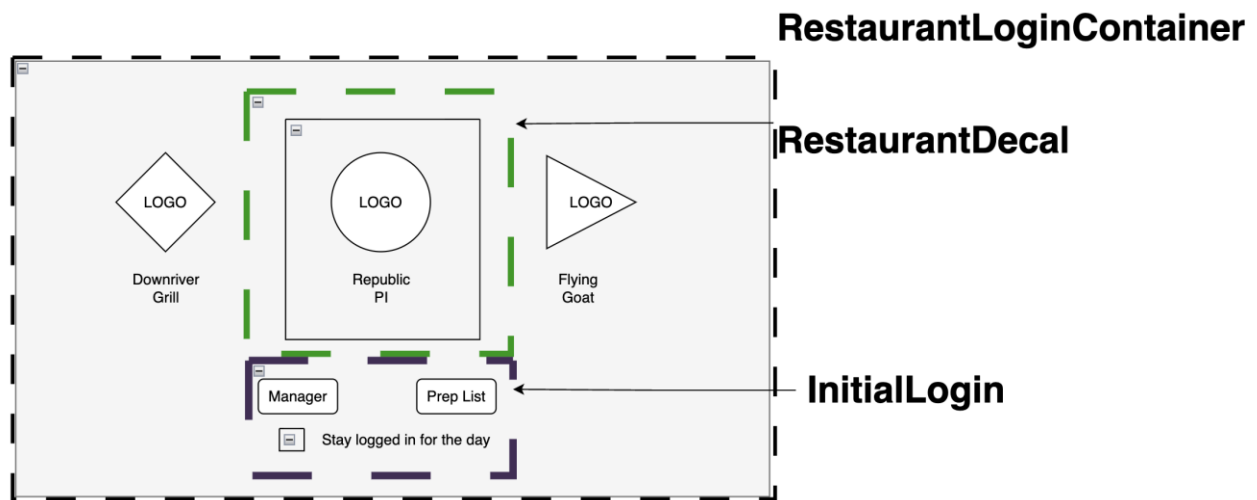
Responsibilities:

Upon the container mounting -

- Gather a list of restaurants too ('fetch Restaurants' method).
- Check if user is already authenticated, if so log them in (Auth Slice).

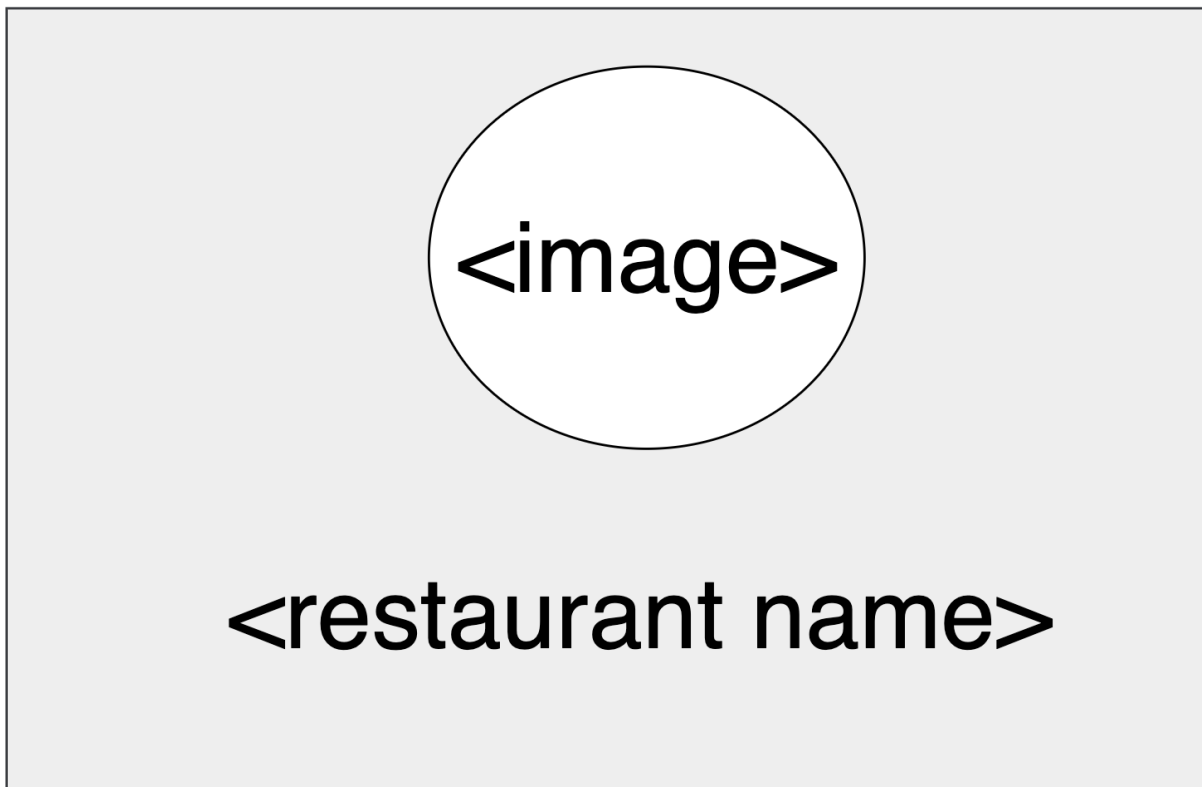
Upon user selecting one (of potentially many restaurants) -

- Manages the state of the selected restaurant based on user selecting a restaurant decal



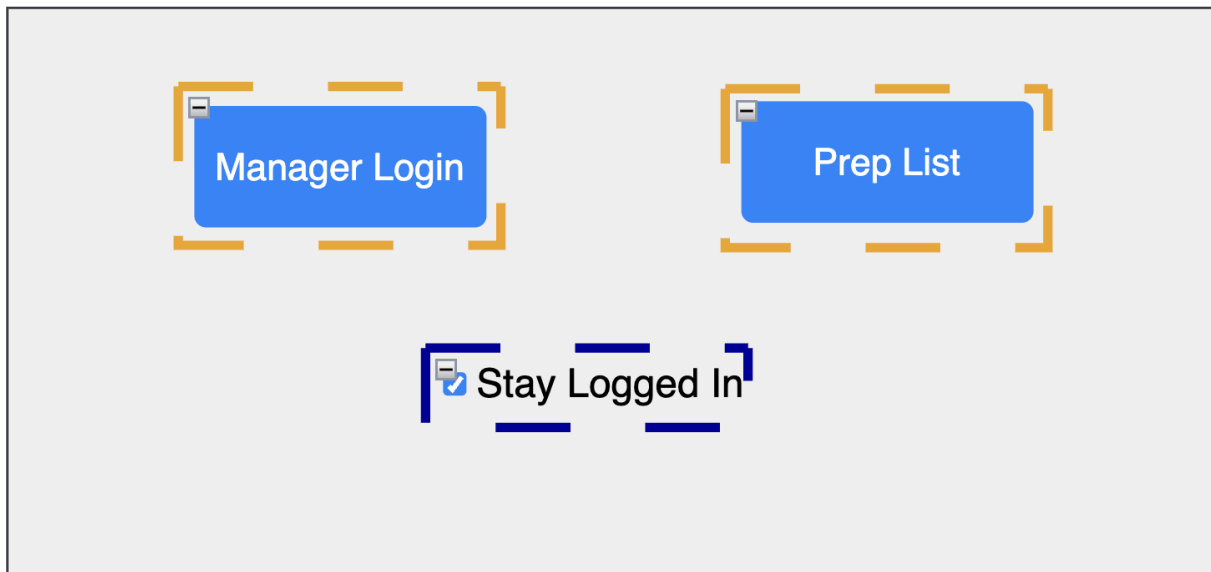
RestaurantDecal:

Reused across the app, mostly in the login stages. Simply displays the image and name for all active restaurants.



InitialLogin:

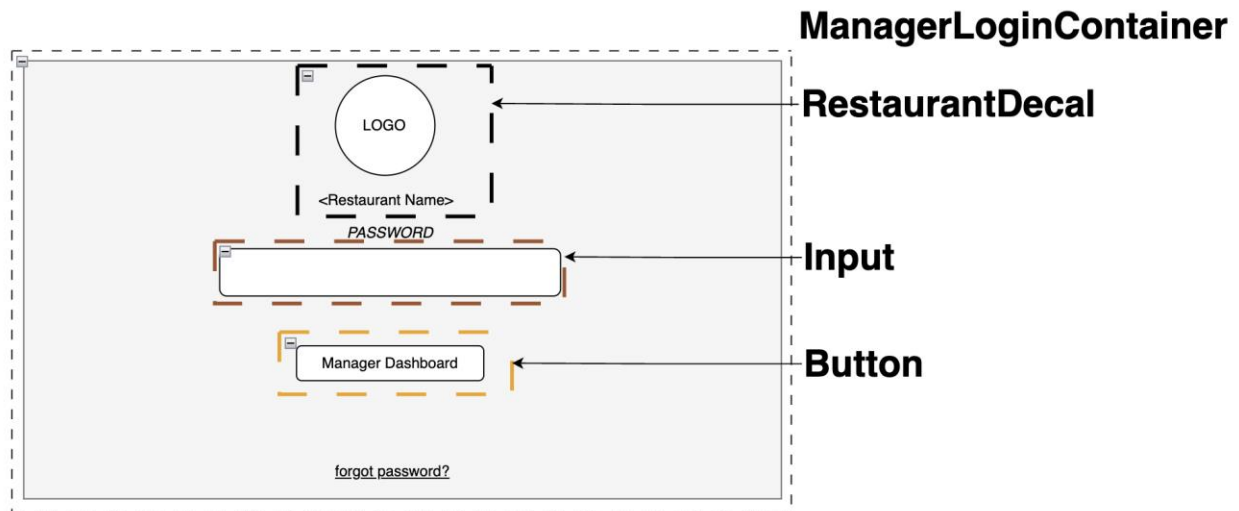
Component specific to this container and only this container. It is made up of two utility buttons and a radio button.



Manager Button	Prep List Button	Stay Logged in Radio Button
Label: "Manager Login" On Click: Passes in onManagerButtonClick() Type: Button Size: Medium	Label: "Prep List" On Click: Passes in onPrepLoginClick() Type: Button Size: Medium	Label: "Stay Logged in" On Change: Passes in onRadioButtonChange() Checked: False Size: Medium

ManagerLogin:

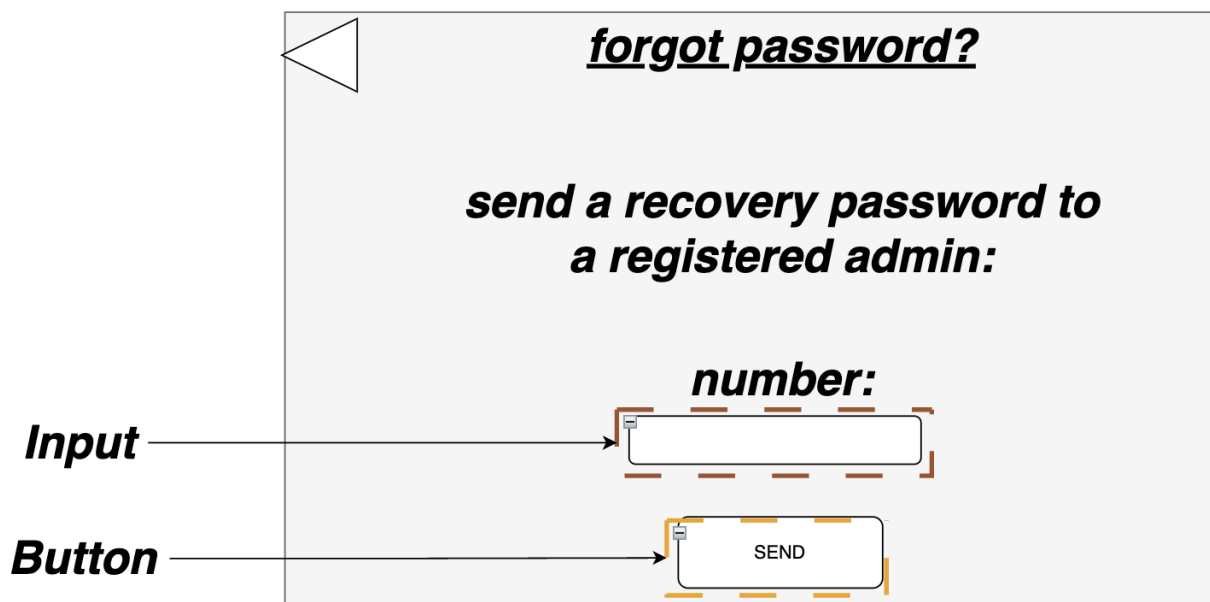
A composition component that manages the password authentication, user state management, JWT creation, and handling login behavior/error handling. Utilizes the restaurant decal defined above, as well as an array of element components whose props are defined above.



Props being passed into the children's components:

Restaurant Decal	Input	Manager Dash Button
Restaurant image: the selected restaurant image Restaurant Name: the selected restaurant name	Placeholder: "password..." Type: text Value: empty On Change: passes in onPasswordChange() Error: "Password is invalid"	Label: "Manager Dashboard" On Click: passes in onManagerDashClick() Type: submit Size: Medium

Forgot Password Container:

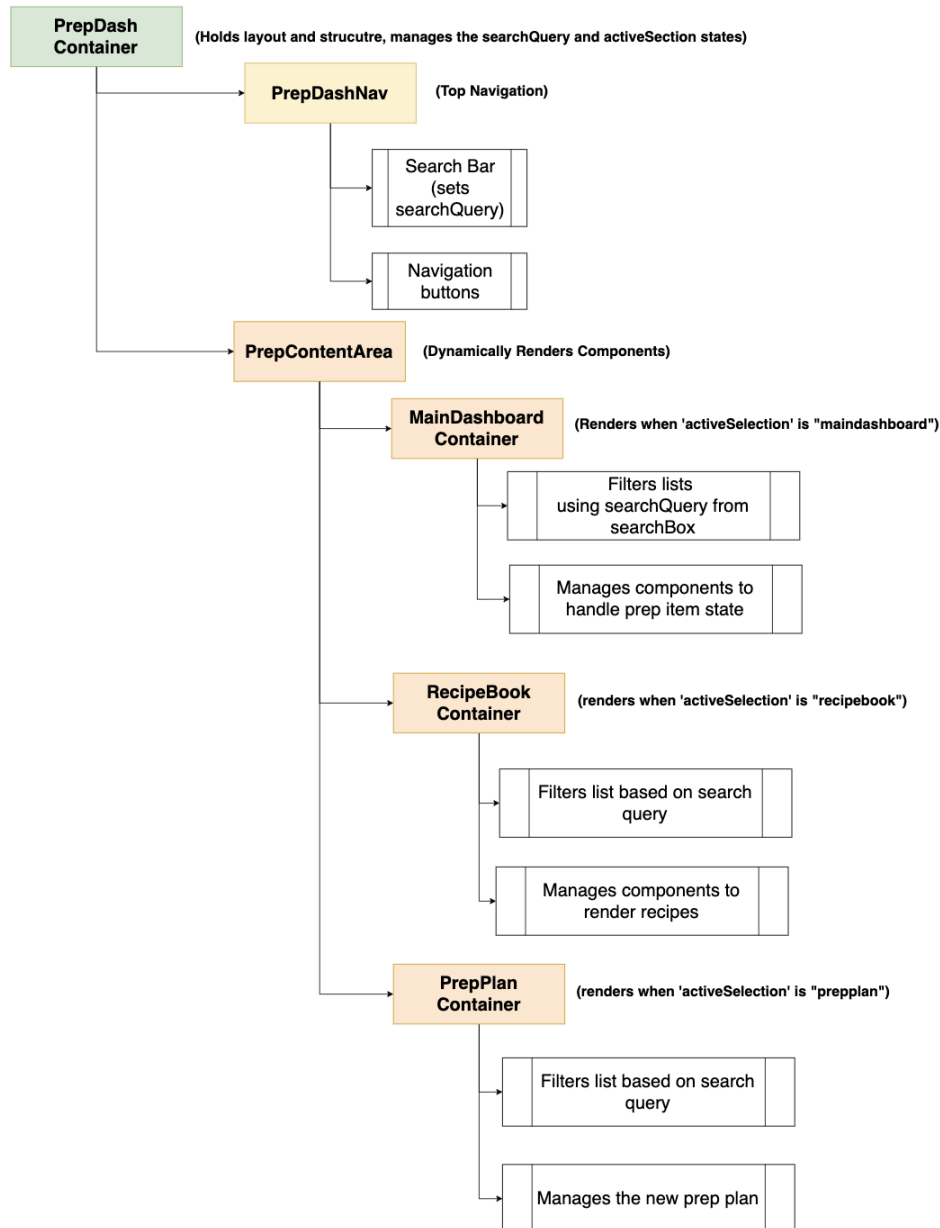


Input	Button
Placeholder: "number..." Type: text Value: none	Label: "Send" On Click: handle Send Click Type: Submit

On Change: set Password function Error: "This number does not exist with an admin. Please Try Again."	Size: Medium
--	---------------------

Prep Dashboard Hierarchy

The following diagrams show the general layout of the prep-dashboard routes as well as some of the higher-level behaviors that each container will handle.



PrepDashContainer:

The prep dash container will always render the nav bar and will dynamically render the container in the content area that is currently selected, this is how the layout looks with its two direct children's components.

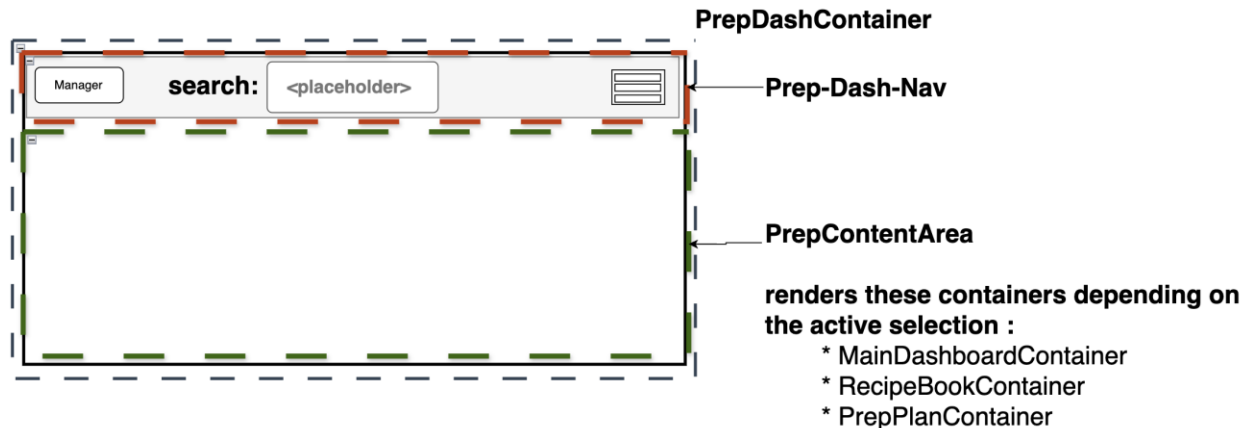
The responsibilities of this container are as follows:

Upon the container mounting:

- Rendering the nav bar and content area with the default selection (which is the main dashboard container)

To successfully render its children's components:

- Passes in the setter of the 'activeSelection' state to the nav component
- Uses the current 'activeSelection' state to render the correct content area component



Prep-Dash-Nav	PrepContentArea
Set Search Query: The function that will set the search query whenever text is typed into the search bar. Set Active Selection: The function that will set the active selection based on user interaction with the side nav. Active Selection: Passes in the current selection so that it can disable that option in the side nav	Search Query: Passes in the current search query so that all lists can be filtered accordingly. Restaurant Id: Passes in the signed in restaurant so the content area containers can render data specific to the restaurant

This container will focus on high level shared functionality that spans across multiple screens within the prep dashboard such as searching content as well as the general layout of the multiple prep dashboard screens. There will be one fixture across all page's display, this is the "Prep-Dash-Nav" component, mounted to the top of the "PrepDashContainer":

Manager Navigation Button	Search Box	Nav Dropdown
Label: "Manager" On Click: the method to handle the click which handles navigation logic Type: Button Size: small	Placeholder: "Search..." Type: text Value: empty On Change: passes in 'setQuery' which updates the current search query. Error Message: none	Options: the list screens available from the prep-dashboard side of the application Selected Value: the current screen the user is trying to navigate away from, called 'activeSelection'.

		On Click: the method that changes the active selection variable
--	--	--

When the user is in the prep dashboard side of the application that nav bar will always be visible. Below it will be a composition component called “PrepContentArea”. This component will handle navigation across the three containers that the user has access to: “PrepMainContainer”, “RecipeBookContainer”, and “PrepPlanContainer”.

Main Dashboard Container

This container is composed of a carousel of buttons and two prep listing components which are defined below.

The responsibility of the Main Dashboard Container is as follows:

Upon the container mounting:

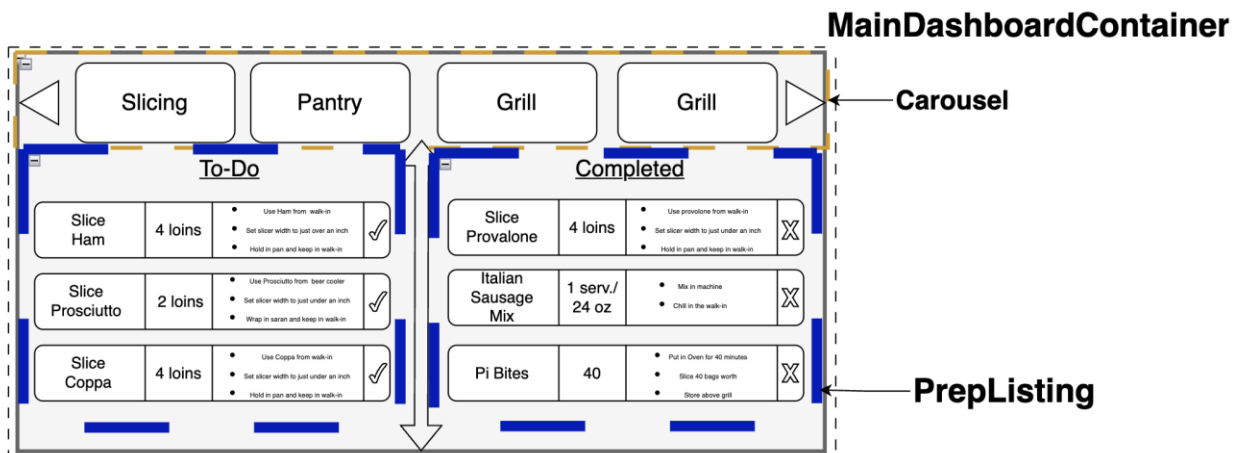
- Fetching data for the days prep list
- Fetching all active categories to filter the list by

Upon a user completing an item:

- Managing the state of Todo and completed item lists

Upon a user cancelling a completed item:

- Managing the state of Todo and completed item lists



Carousel	Prep Listing
items: passes in the fetched categories which describe prep items Button Width: 2rem's (subject to change) Button Height: 3rem's (subject to change) On Button Click: set the active category using the 'setActiveCategory' variable Visible Buttons: 4 buttons visible at one time	List: passes in the list to be displayed On Button Click: the function which handles a completion or cancelling a completion of an item Label: "To-Do" for the left list and "Completed" for the right list. Category: If a category is selected, will filter the list

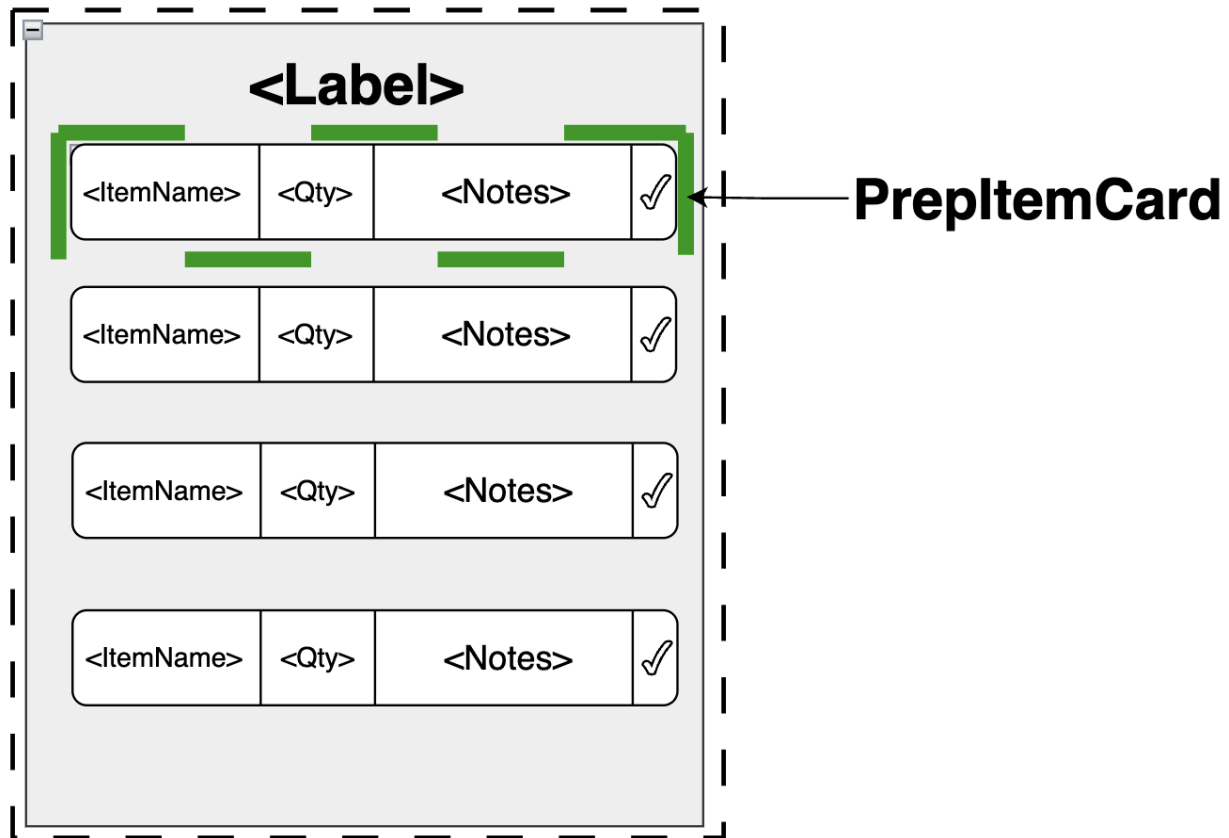
Prep Listing:

Displays a list of 'Prep Item Card' components that is defined below

The responsibility of the Prep Listing is as follows:

Upon the component mounting:

- Filters the list based on category depending on if a selected category is passed in
- Upon the selected category being updated
- Filters the list based on category
- To successfully render its child component:
- Pass in one of the list items with either the 'handleComplete' or 'handleCancel' function



Prep Item Card

The responsibility of the Prep Item Card is as follows:

Upon the component mounting:

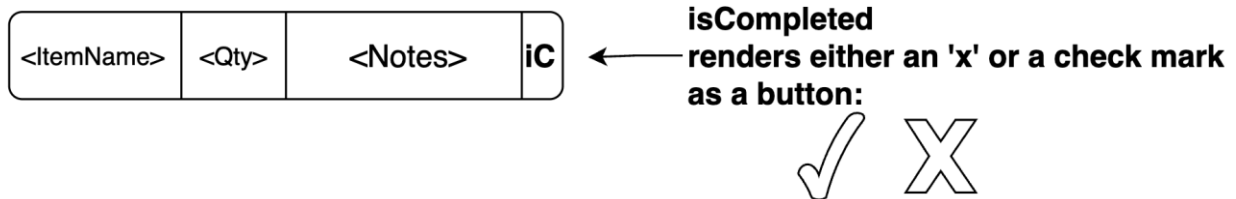
- Sets is completed state based on what list it's in

Upon the user completing a to do item:

- Handles the complete with the specific Item Id

Upon the user cancelling an item:

- Handles the cancel with the specific item id



Recipe Book Container

The responsibility of the Recipe Book Container is as follows:

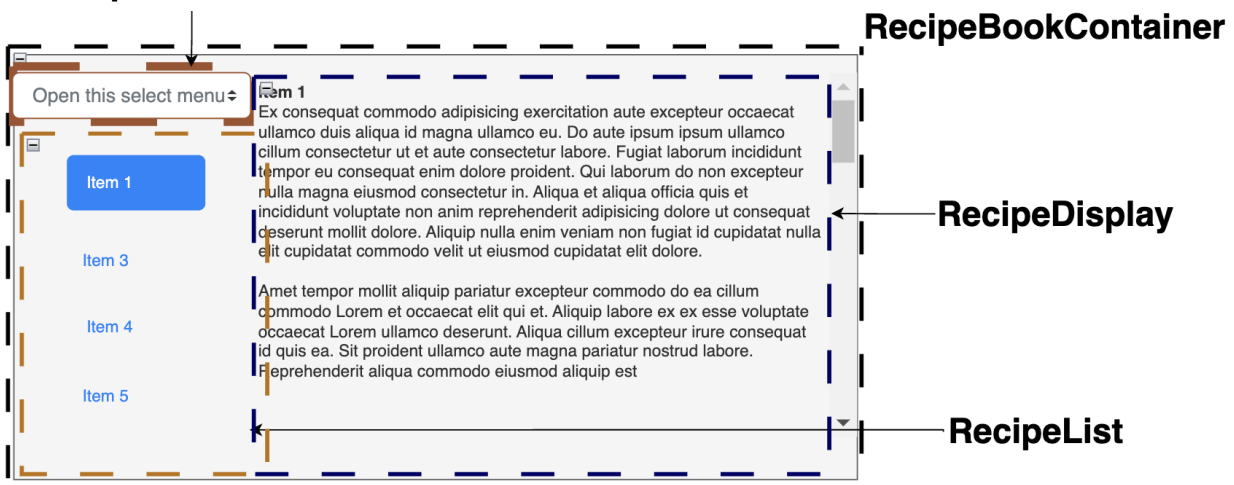
Upon the container mounting:

- Fetch all active recipes
- Fetch all categories for prep list

Upon a Recipe list being selected:

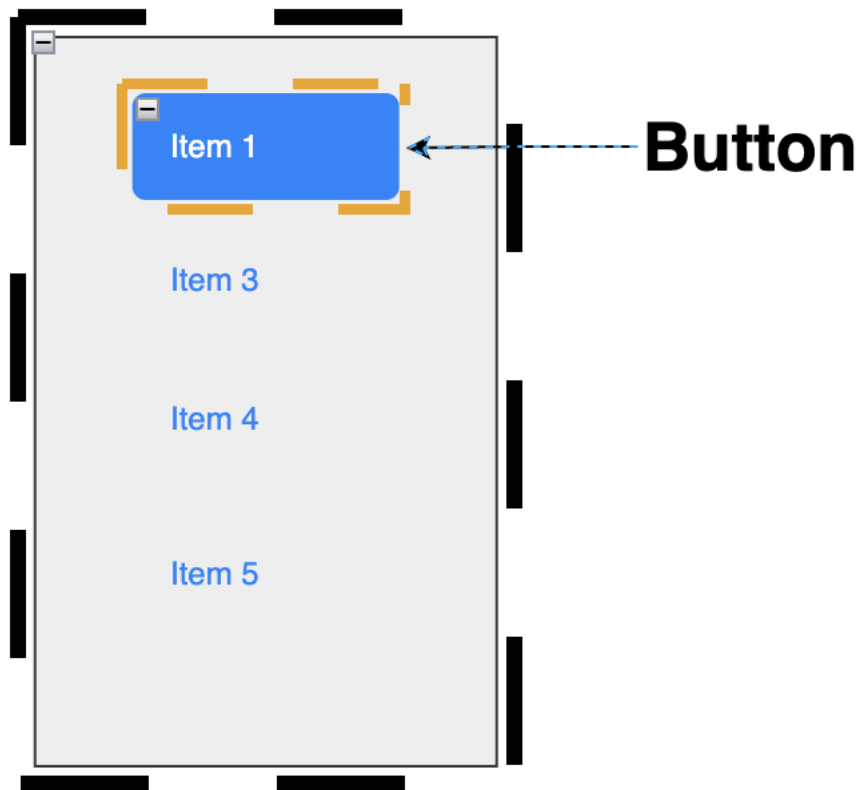
- Fetch the data associated with the selected recipe
- Set the active recipe

DropdownMenu



Recipe Display	Recipe List	Dropdown Menu
Passes in the Recipe instructions to display.	Passes in the list of recipes to assign each to a button. Will not require any design since it is only displaying.	Items: passes in the categories fetched on mount Placeholder: "categories..." Selected Value: none On Select: passes in 'setCategory' to update the active category

Recipe List:



Prep Plan Container

This container is composed of a dropdown menu, an all-prep items list and a prep listing for daily plan items.

The responsibility of the Prep Plan Container is as follows:

Upon the container mounting:

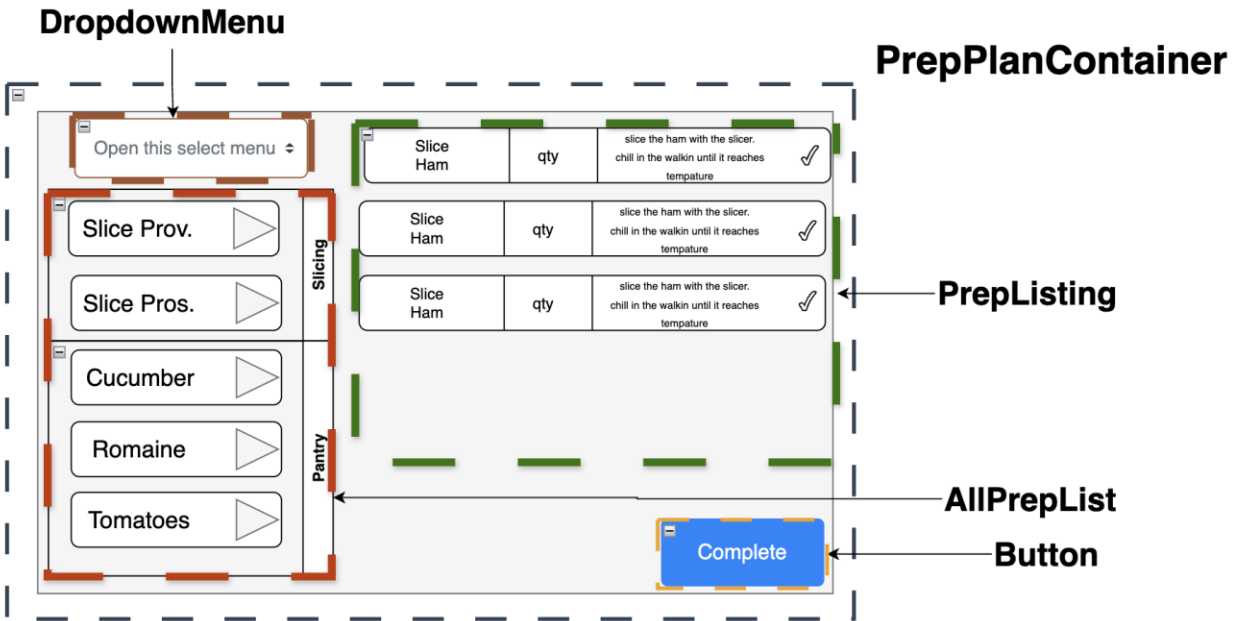
- All active prep items are fetched for the logged in restaurant
- All Categories are fetched

Upon the user adding a prep item to the next day's prep plan:

- Update both all prep items and daily plan list

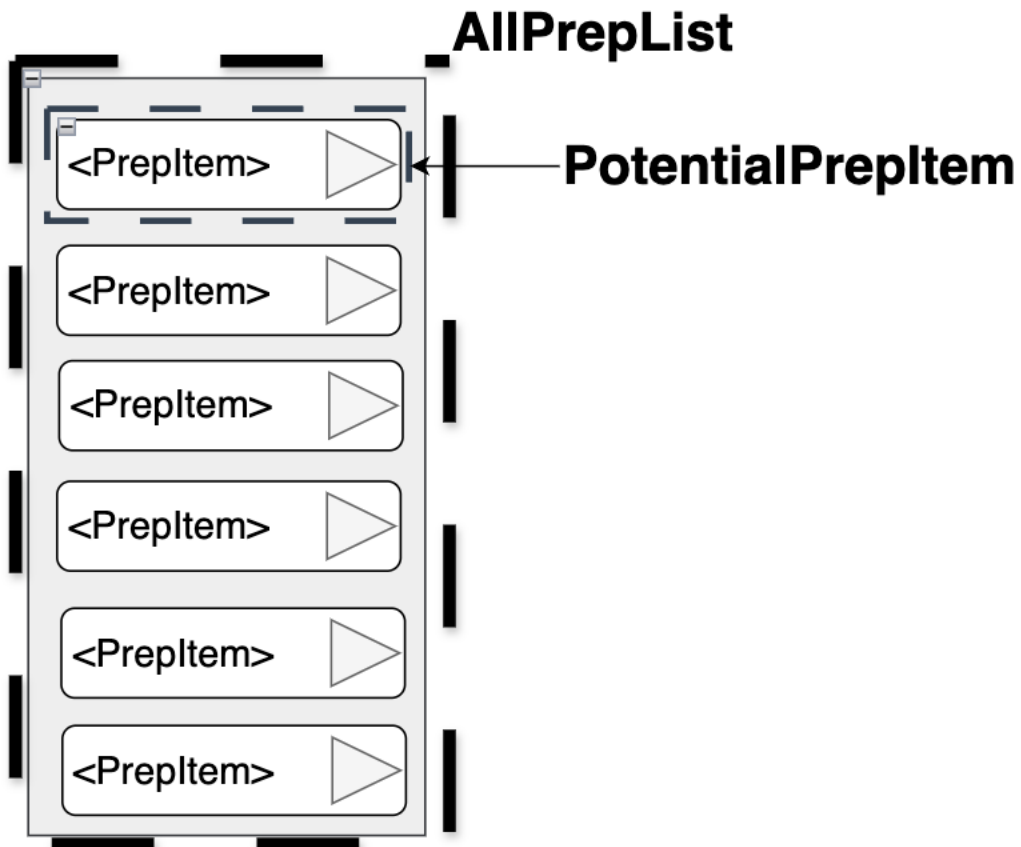
Upon the user updating the quantity on a daily plan item

- Update the daily plan item
 - Refresh the list



Dropdown Menu	Prep Listing (View design above)	All Prep List	Button
Items: passes in the categories fetched on mount Placeholder: "categories..." Selected Value: none On Select: passes in 'setCategory' to update the active category	List: the prep items that have been added to the daily prep list Handle Complete: no function needed. Handle Cancel passes removeFromDailyPrep() function Category: passes in the category variable Label: completes to indicate that the buttons should be 'X'.	List: All prep items that can be added to a daily prep list	Label: "Complete" On Click: handleDailyPrepSubmit() Type: Submit Size: Medium

All Prep List:



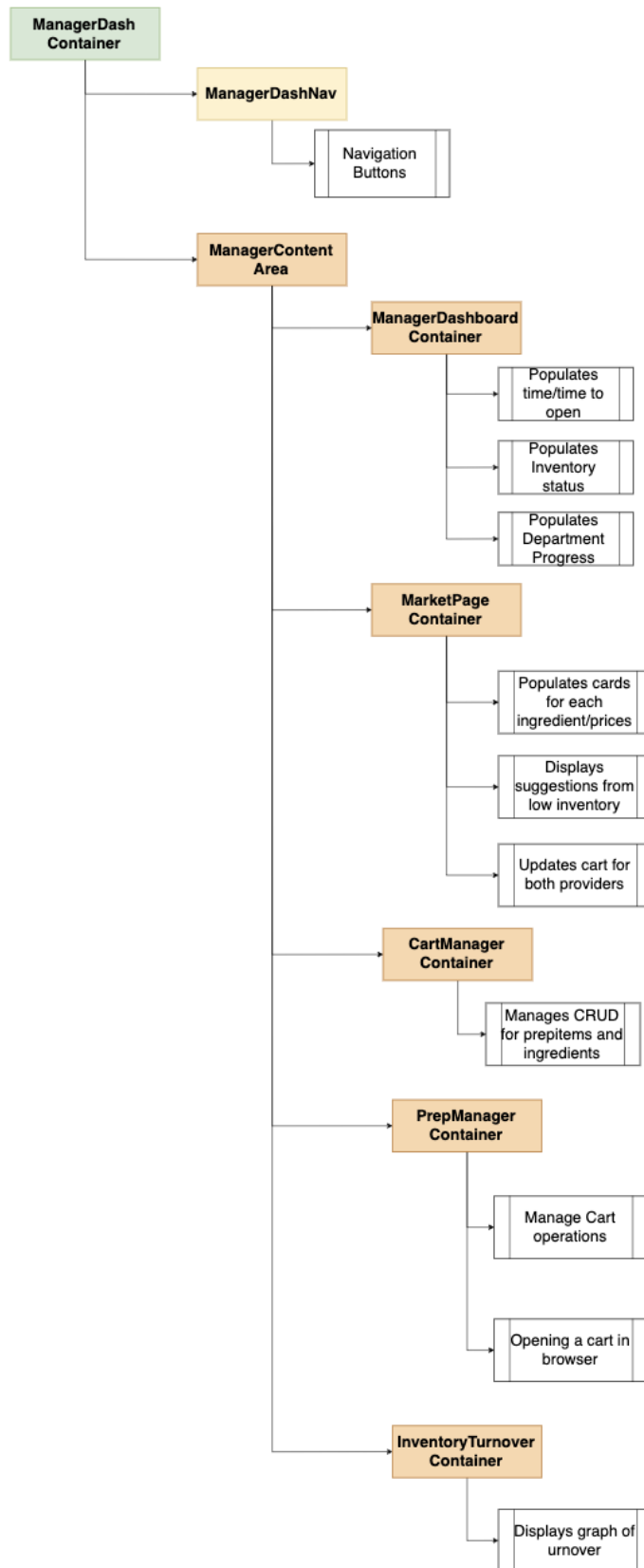
Potential Prep Item

Name: Name of the potential prep item.

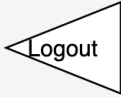
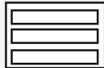
Handle Click: the method to handle a click within one of the potential prep items

Manager Dashboard Hierarchy

The following diagrams show the general responsibilities and structure of the manager-dashboard routes.

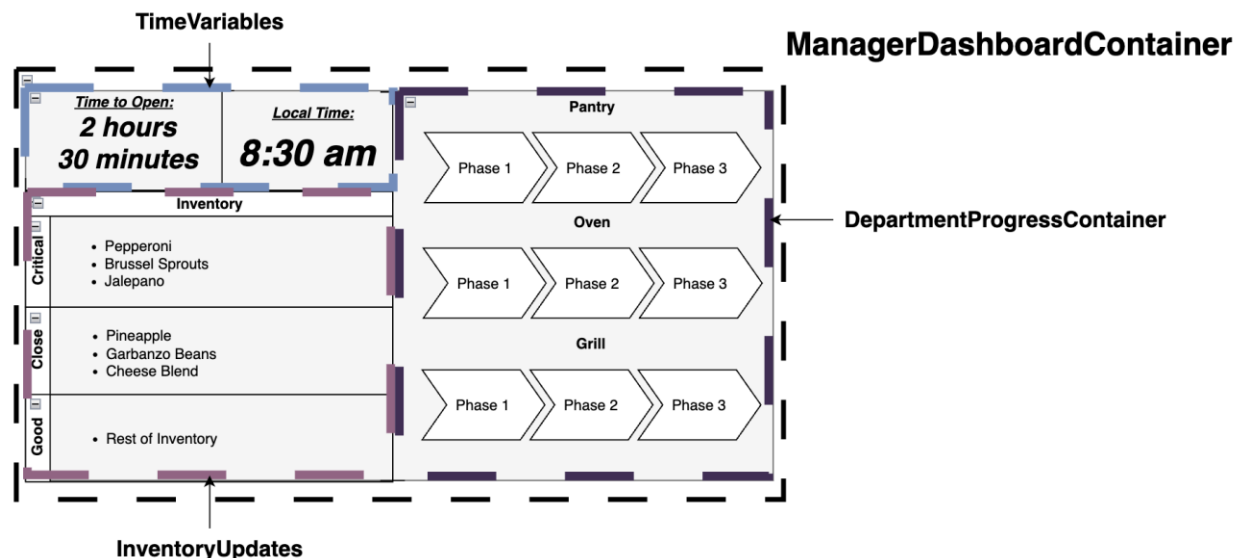


In the same fashion that the prep-dashboard routes follow, all content will be contained by a “ManagerDashContainer”. This container will focus on managing navigation between the multiple manager dashboard screens. There will be one fixture across all containers display, this is the “Manager-Dash-Nav” component, mounted to the top of the “PrepDashContainer”:

	
Logout button Logs out the signed in manager once a modal is populated confirming that the manager would like to sign out. This action deletes the session data so no one can access the manager dashboard without logging in first	Nav Dropdown Options: the list of options Current Screen: the current screen the user is trying to navigate away from On Click: the method that handles a dropdown option selection

Manager Dashboard Container

No props need to be passed in since all strictly depend on what the signed in restaurant is to internally populate their children's components.



Time Variables (lives within the Manager Dashboard Container)

The responsibility of this component is as follows:

Upon the component mounting:

- Fetches the opening time according to the logged in restaurant
- Sets the current time and time until open

Upon a change in one of the time states:

- Updates the time and time until open

Inventory Updates

(lives within the Manager Dashboard Container)

The responsibility of this component is as follows:

Upon the component mounting:

- Fetches all ingredients specific to the logged in restaurant
- Filters each ingredient into correct list (critical, close, good)

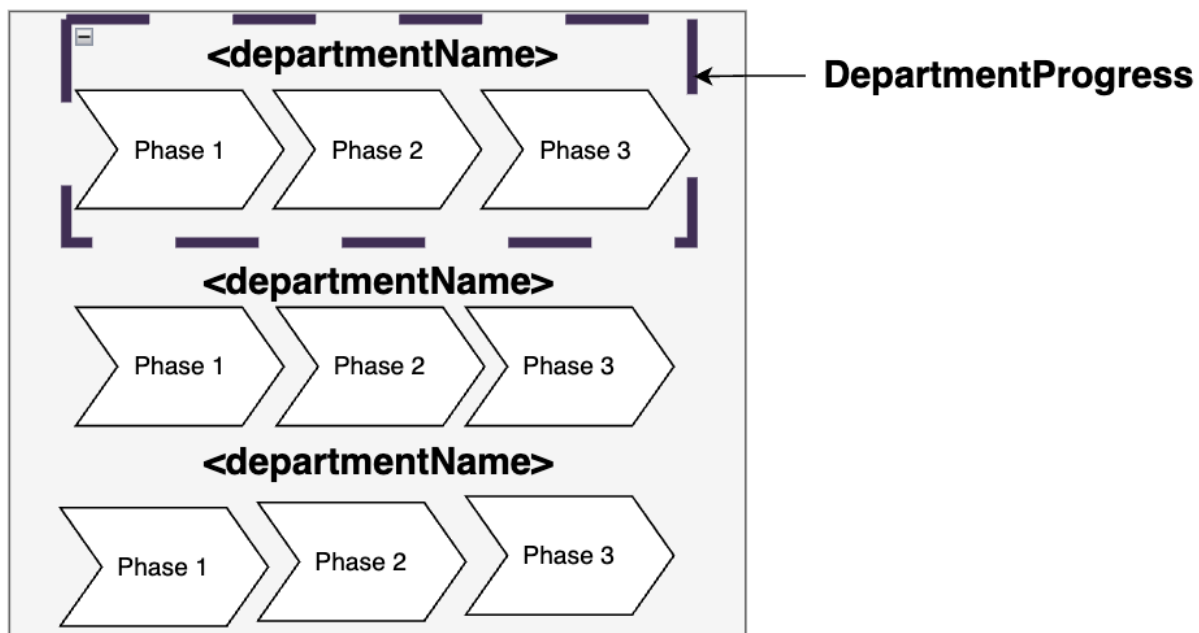
Department Progress Container

(lives within the Manager Dashboard Container)

The responsibility of this component is as follows:

Upon the container mounting:

- Fetches the departments specific to the logged in restaurant



Department Progress

Department Name: Passes in the name that is displayed above the progress bar

Department Progress: Passes in a percentage of completion at the given time of the data being fetched.

Market Page Container

The responsibility of the Market Page Container is as follows:

Upon the container mounting:

- Fetches all ingredients specific to a restaurant

Upon the content being searched using the search bar:

- Updates the searchQuery state

Provider Market

Searchbar

Input

Today's Suggestions:

Cucumbers

last ordered: 12/23

Sysco: 7.00 \$

Us Foods: 7.50 \$

QTY 3

select

select

+

-

A

D

D

Pecorino

last ordered: 12/23

Sysco: 7.00 \$

Us Foods: 7.50 \$

QTY 3

select

select

+

-

A

D

D

Olives

last ordered: 12/23

Sysco: 7.00 \$

Us Foods: 7.50 \$

QTY 3

select

select

+

-

A

D

D

Category Selector

Pineapple

last ordered: 12/01

Sysco: 7.10 \$

Us Foods: 6.50 \$

QTY 0

select

select

+

-

A

D

D

Pineapple

last ordered: 11/25

Sysco: 2.12 \$

Us Foods: 2.18 \$

QTY 0

select

select

+

-

A

D

D

Beets

last ordered: 12/14

Sysco: 3.00 \$

Us Foods: 4.50 \$

QTY 0

select

select

+

-

A

D

D

Eggs

last ordered: 12/04

Sysco: 8.00 \$

Us Foods: 7.90 \$

QTY 0

select

select

+

-

A

D

D

Mushrooms

last ordered: 12/20

Sysco: 2.00 \$

Us Foods: 1.50 \$

QTY 0

select

select

+

-

A

D

D

Brussel Sprouts

last ordered: 12/18

Sysco: 3.00 \$

Us Foods: 4.50 \$

QTY 0

select

select

+

-

A

D

D

DailySuggestions

IngredientMarketList

Input	Daily Suggestions	Ingredient Market List
Placeholder: "Search..." Type: text Value: empty On Change: passes in the HandleSearchChange() method	No Props	No Props

Daily Suggestions

The responsibility of the Daily Suggestions container is as follows:

Upon the container mounting:

- Fetches all critical items to be displayed

To successfully render its children's components:

- It will pass down the 'updateCart' method
- It will pass in one of the ingredients names

Ingredient Market Card

The responsibility of the Ingredient Market Card is as follows:

Upon the container mounting:

- Fetches the ingredients details to be displayed

Upon the user selecting a radio button under a specific provider:

- Updates the selected provider state

Upon the user modifying the quantity of an ingredient:

- Updates the quantity state accordingly

© 2020. Grand Canyon University. All Rights Reserved.

38

Upon the user adding an ingredient to the cart:

- Updates the cart state using the passed in 'updateCart' method

Prep Manager Container

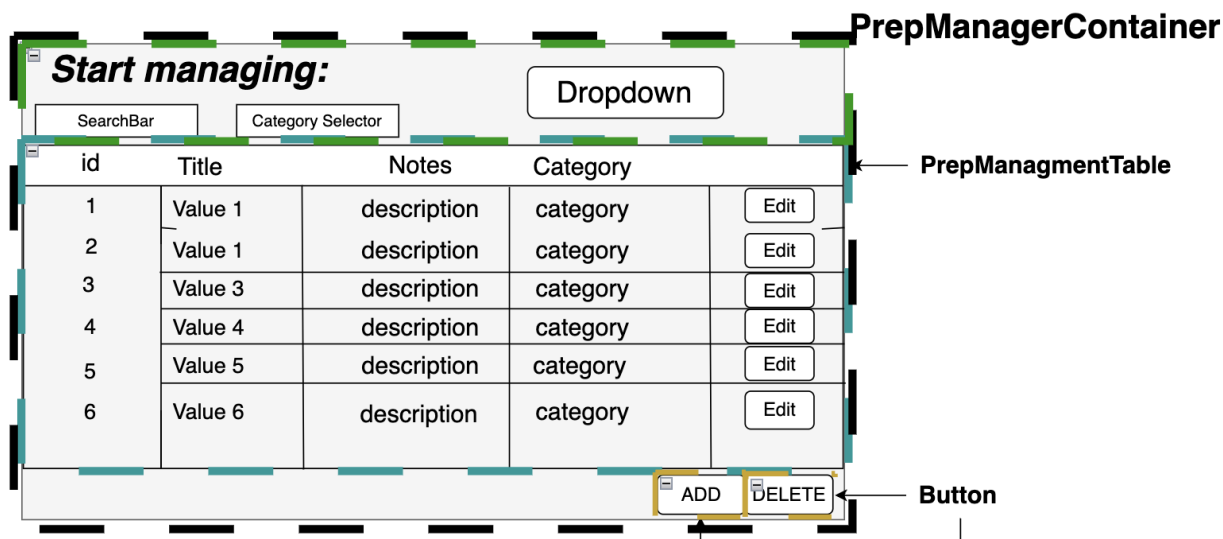
The responsibility of the Ingredient Market Card is as follows:

Upon the container mounting:

- Fetch the categories for prep items and ingredients

Upon the user Selecting Prep Item or Ingredients:

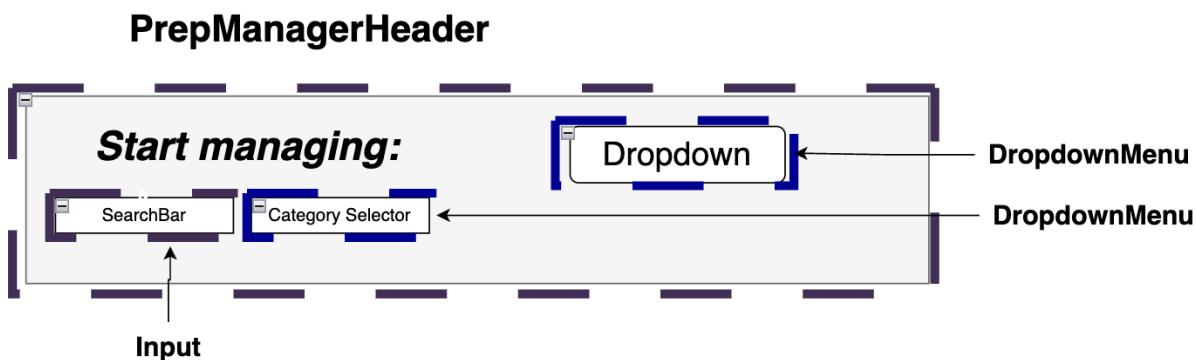
- Pass the active selection into the table



Prep Manager Header:

The responsibility of the Prep Manager Header is as follows:

Set the values of all the search, category, and prep item or ingredients upon being selected in one of the three children's elements.



Search Bar	Category Dropdown	Prep dropdown
Placeholder: "Search..." Type: text Value: empty On Change passes in the method to set the search query	Options: the categories that sort the items Placeholder: "Categories..." Selected value: None	Options: the options, "prep items" or "ingredients" Placeholder: "Mange..." Selected value: None

Error Message: none	On Select: Passes in the method to handle a category being selected	On Select: Passes in the method to handle a category being selected
----------------------------	--	--

Prep Management Table:

id	Title	Notes	Category	
1	Value 1	description	category	Edit
2	Value 1	description	category	Edit
3	Value 3	description	category	Edit
4	Value 4	description	category	Edit
5	Value 5	description	category	Edit
6	Value 6	description	category	Edit

← **ListItem**

Each list item requires these values to be passed in:

<ID>	<Name>	<Notes>	<Category>	Edit
------	--------	---------	------------	------

Cart Manager Container:

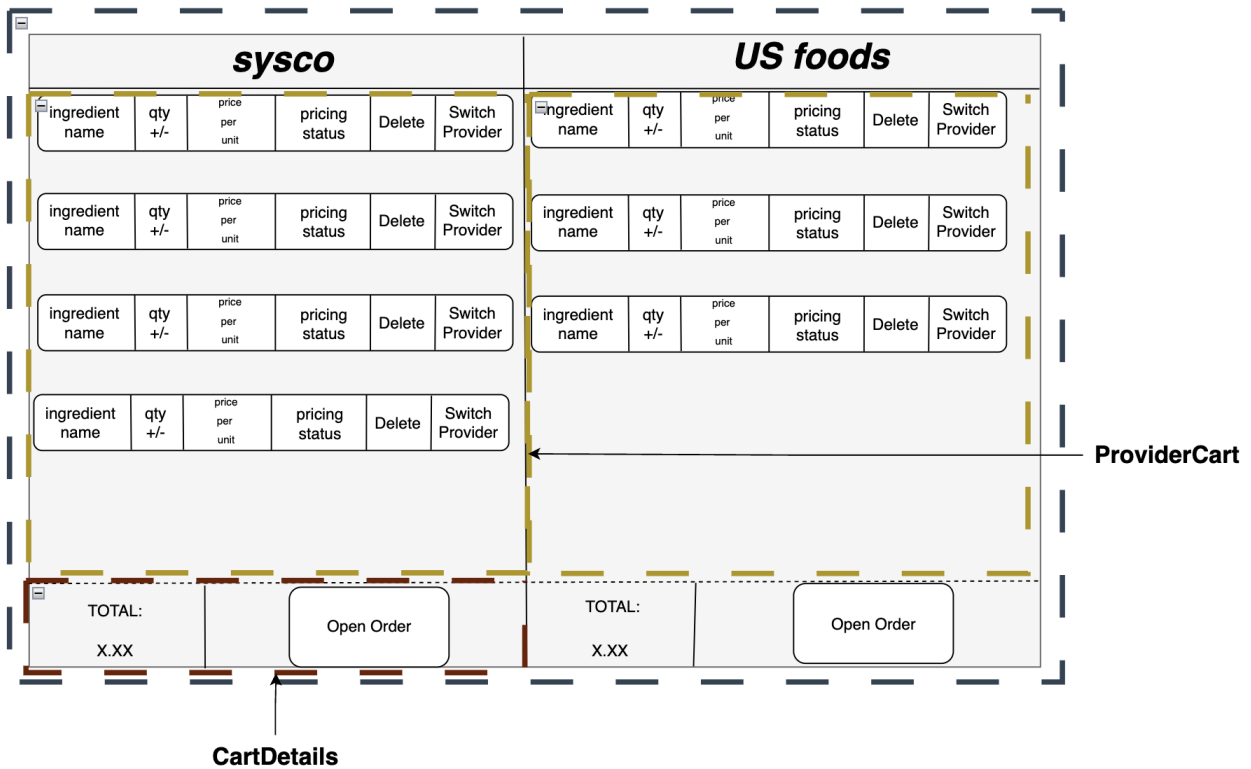
The responsibility of the Cart Manager Container is as follows:

Upon the container mounting:

- Fetching both carts for each provider

Upon a cart changing:

- Updating the totals to be passed into the Cart Details card

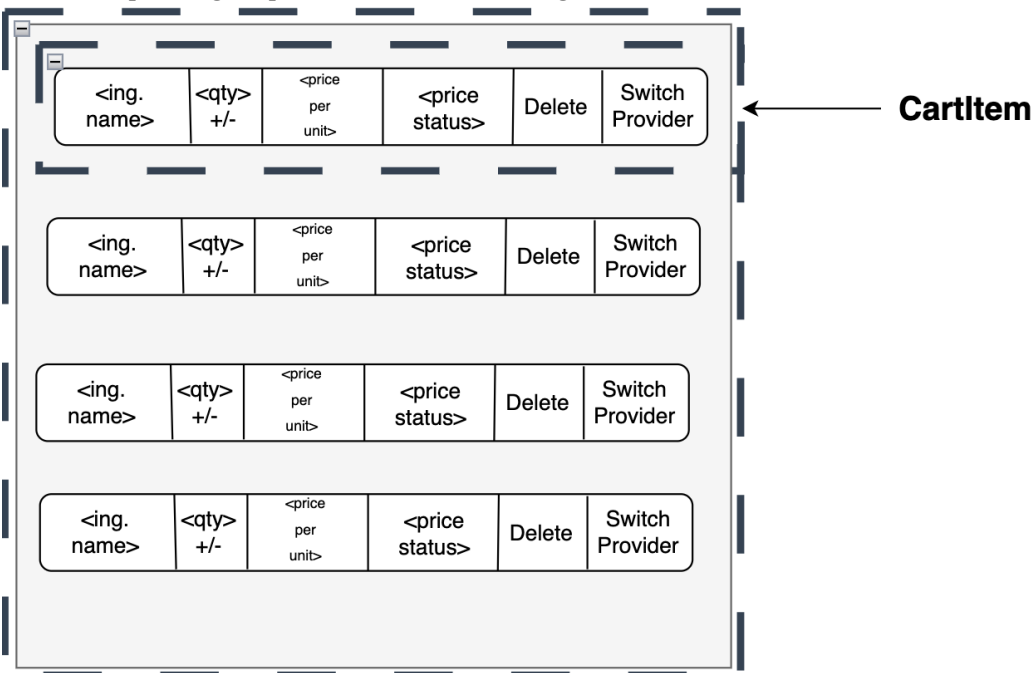


Provider Cart:

The responsibility of the Cart Manager Container is as follows:

Upon the container mounting:

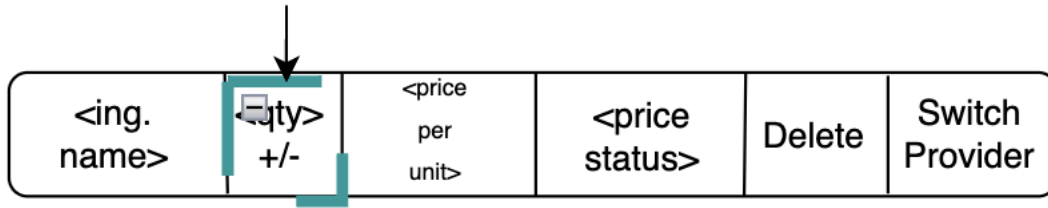
- Populating all passed in cart items using the Cart Item card



Cart Item:

Requires these following props:

Number Select

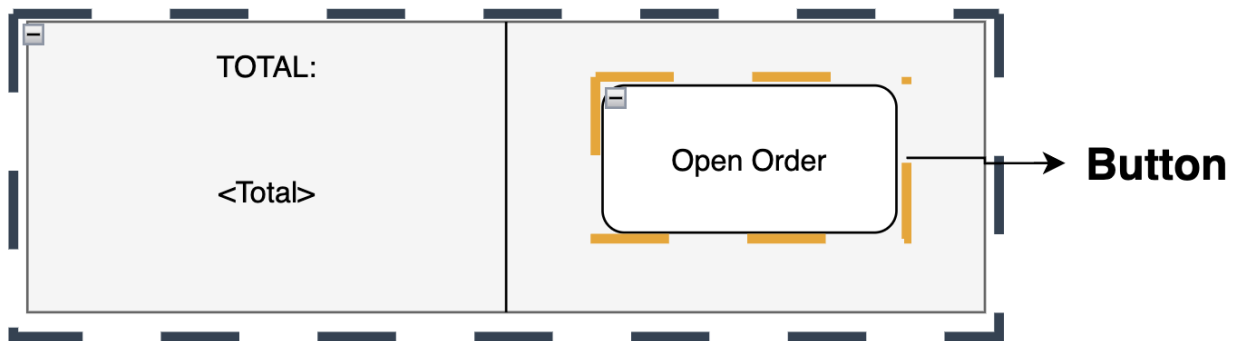


Number Select
Value: "{qty}" On Change: method that handles a change of number Min: 0 Max: 0 Step: 0.5 Label: "QTY"

Cart Details:

The responsibility of the Cart Manager Container is as follows:

- Displaying the total price and "Open Order" button



Button
Label: "Open Order" On Click: method to handle click behavior Type: submit Size: Medium

Inventory Turnover Visualizer:

The responsibility of the Turnover visualizer is as follows:
Populating the I Frame from tableau.

UML Diagrams:

You should insert any UML Class diagrams and UML Sequence diagrams here. If you have no supporting documentation, explain the rationale why you are able to leave this section as N/A.

Frontend:

Before listing the component and state management UML's, here is a simple "API service" that will be utilized in some of the screens:

 ApiService
- BASE_URL : string
+ fetchCategories(type : string) : string[] + fetchRestaurants() : Restaurant[] + fetchManagers(restaurantId: number) : Manager[] + fetchRecipes(restaurantId : number): Recipe[]

Within the frontend, all data handled within the context of global state utilizes ‘Slice’ modules which handle centralized state management, actions upon state, and so on. Here are how those classes will be designed:

Authentication:

Because the current logged in restaurant Id and user role will be used in almost all screens of the application, this will be managed globally.

 AuthSlice
- isAuthenticated : bool - restaurantId: number - userRole : string - authToken : string
+ login(payload: AuthSlice) : void + logout() : void + updateRestaurantID(payload: int) : void + updateUserRole(payload: string): void

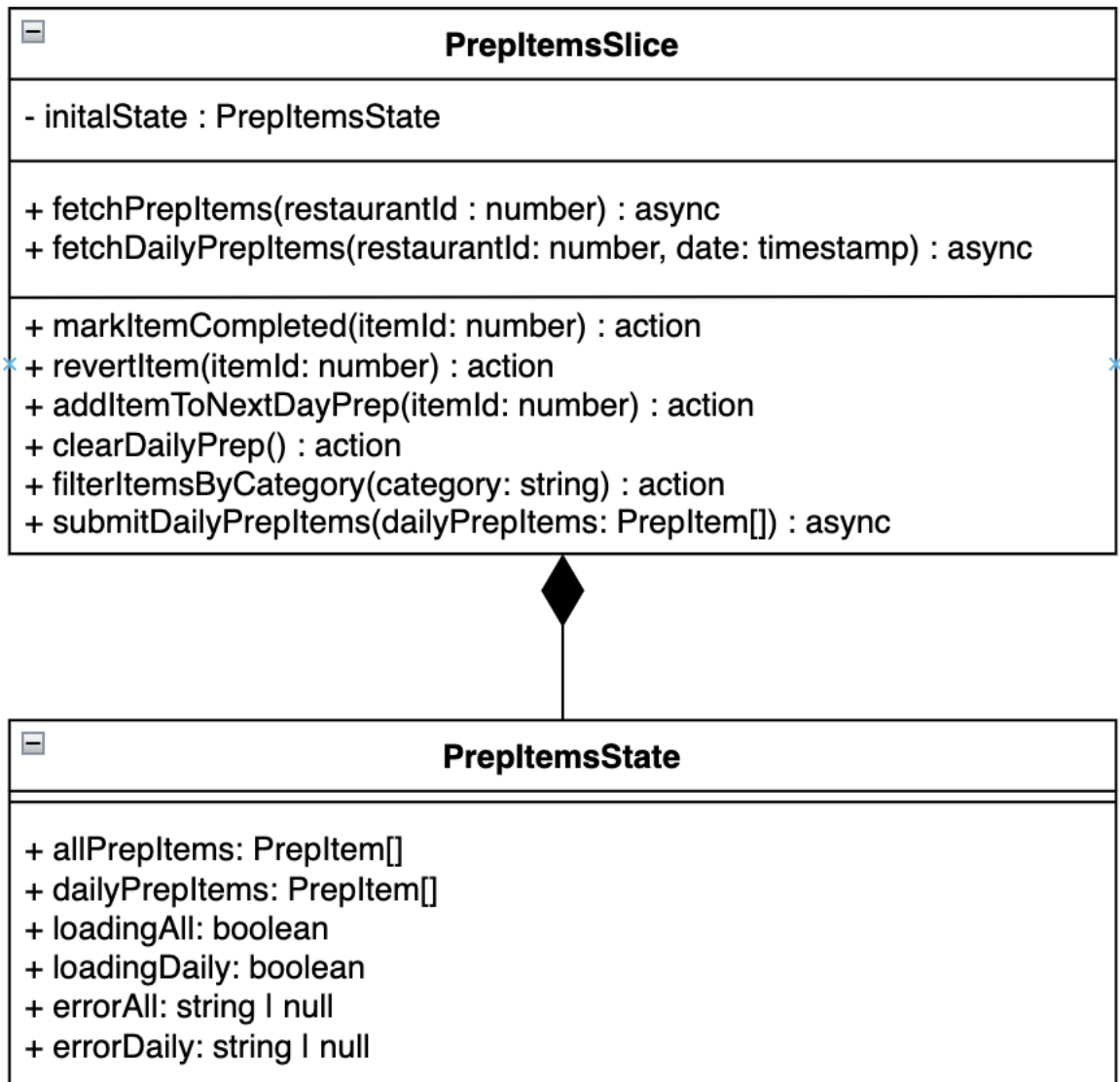
Search queries:

Search queries are simply handled by separating the search queries of the separate dashboards, one for the prep dashboard screens and one for the manager dashboard screens.

 SearchSlice
- managerSearchQuery : string - prepDashboardSearchQuery : string
+ setManagerSearchQuery(query: String) : void + setPrepSearchQuery(query: String): void

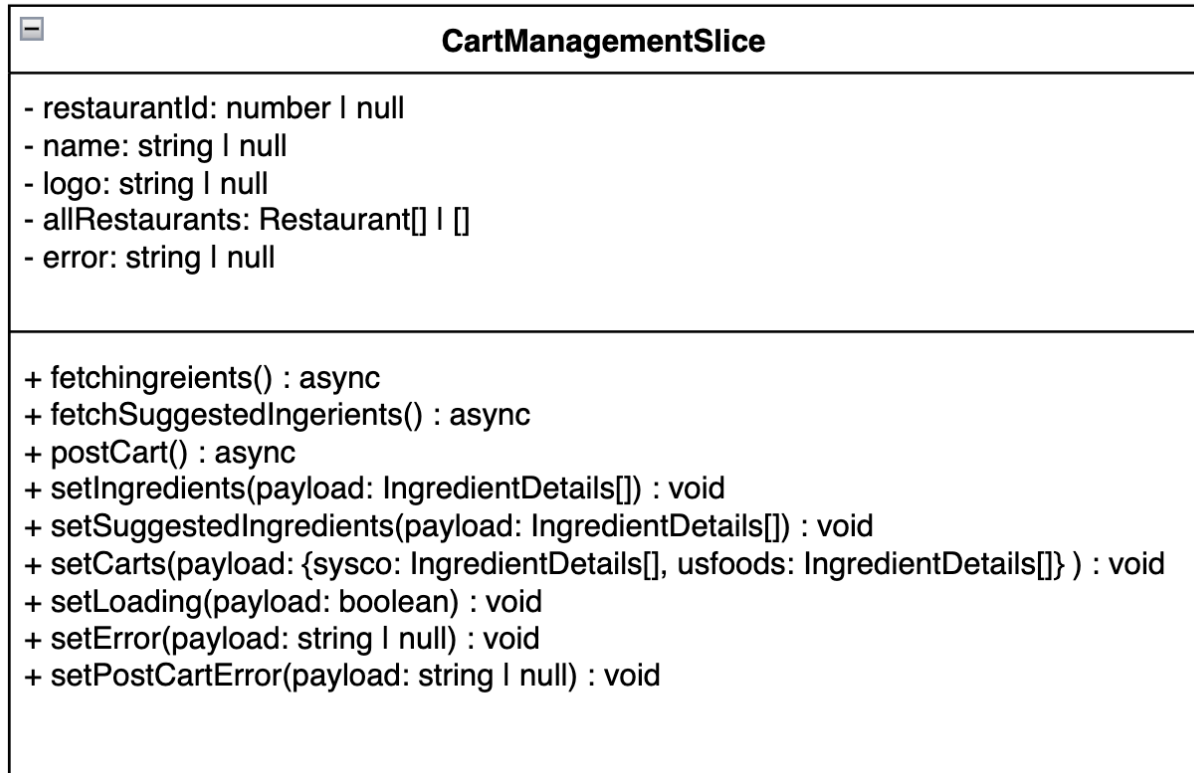
Prep Items/Lists:

Prep Items/Lists will be managed globally because there are a few screens that will use the same prep list data. It will also prove beneficial so there is less posting to the backend. If it were stored locally in some of the key components this would require a post upon all changes to data in order to keep the data fresh in other screens.



Cart Management:

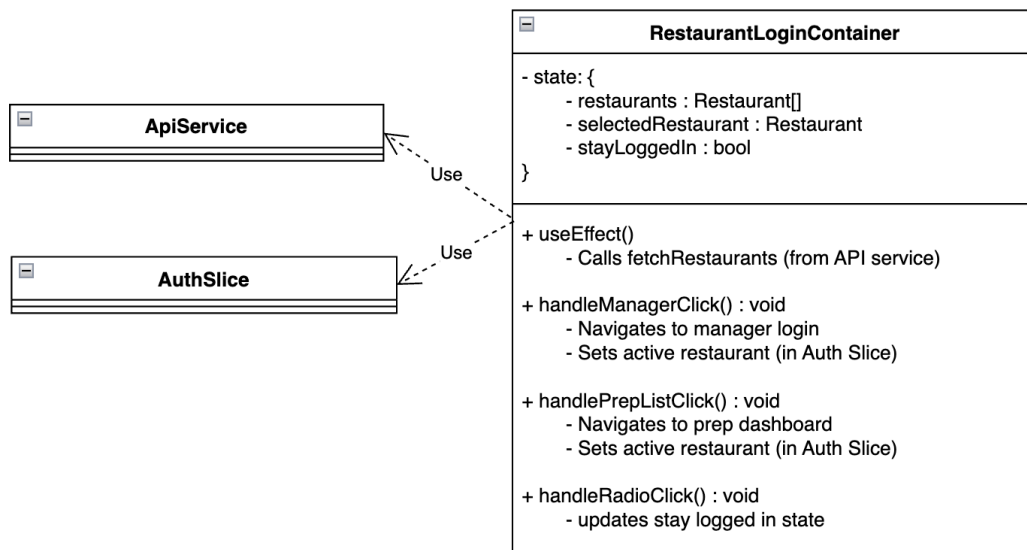
It is crucial that cart management is handled globally because there likely will be a fair number of modifications in either cart or this must be consistent across all screens.



Component Design:

Restaurant Login Container:

Handles navigation buttons behaviors, fetching restaurant data, and setting the active restaurant upon navigation.

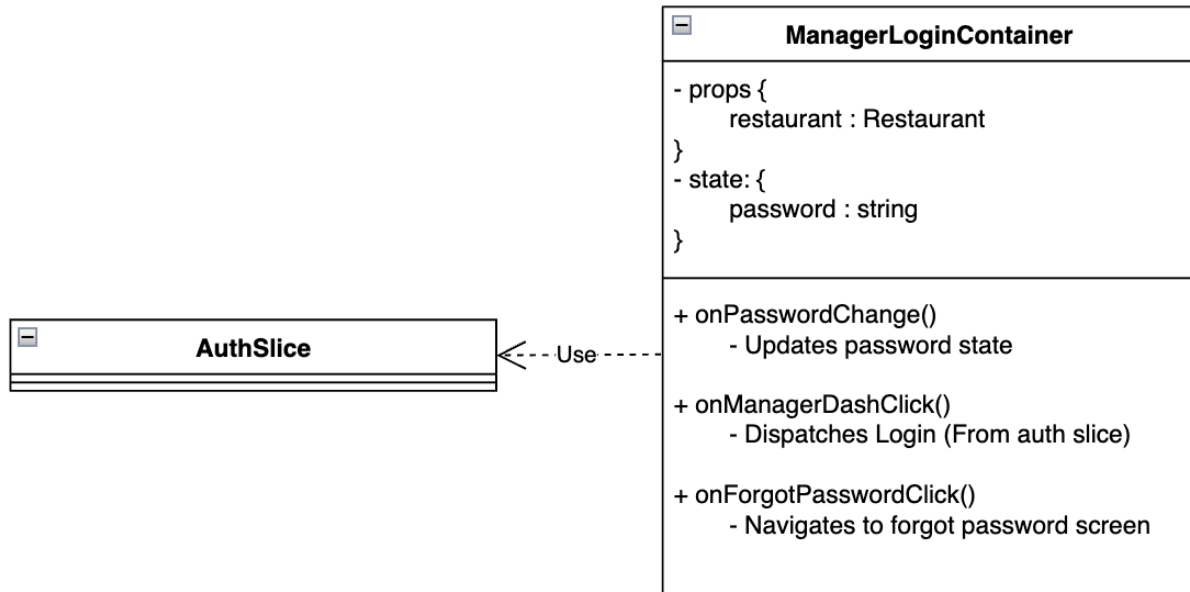


Restaurant Decal and Initial Login:

Simply displays the restaurant name and image. Handles clicks and switches of all buttons and radio buttons.

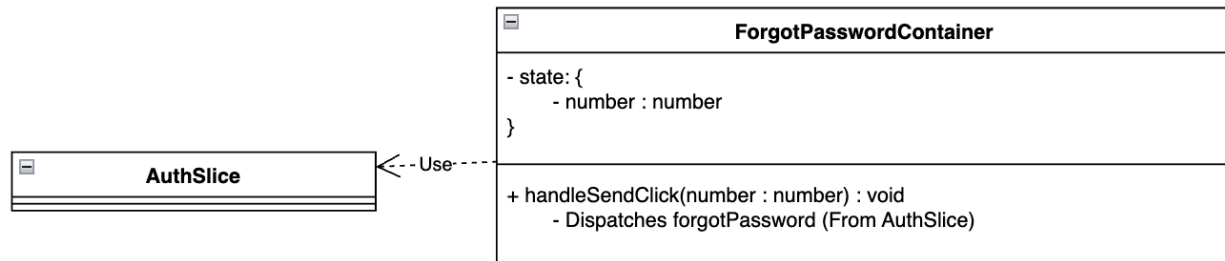
Manager Login Container:

Handles login logic and navigates to manager dashboard or forgot password screen.



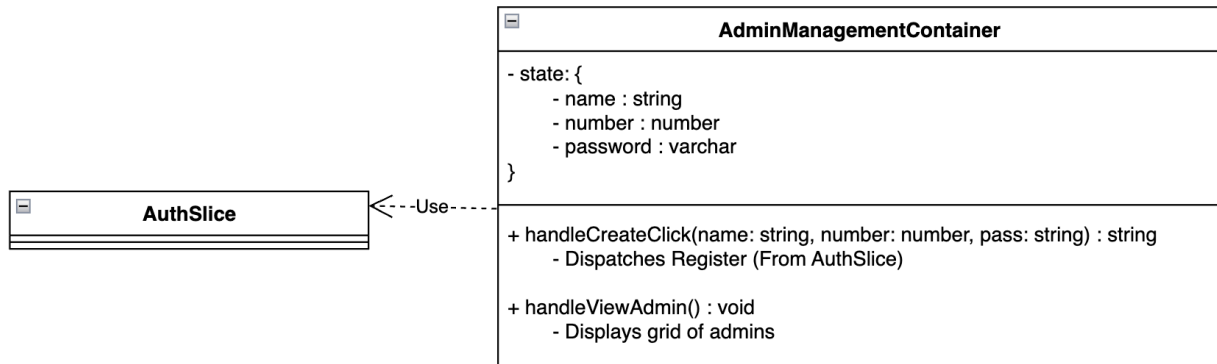
Forgot Password Container:

Handles the process for resetting a password.



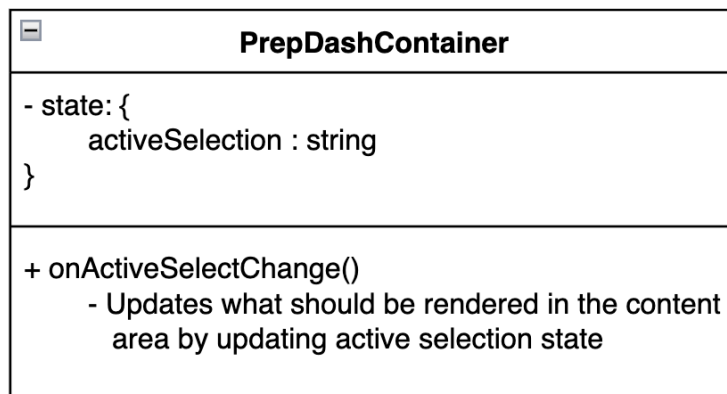
Admin Management Container:

Handles creating a new admin user, and behaviors of viewing all admins.



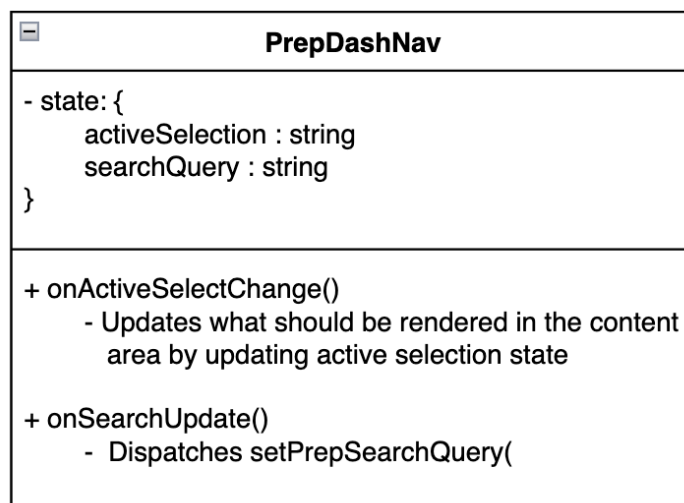
PrepDashContainer:

Handles updates of the content area.



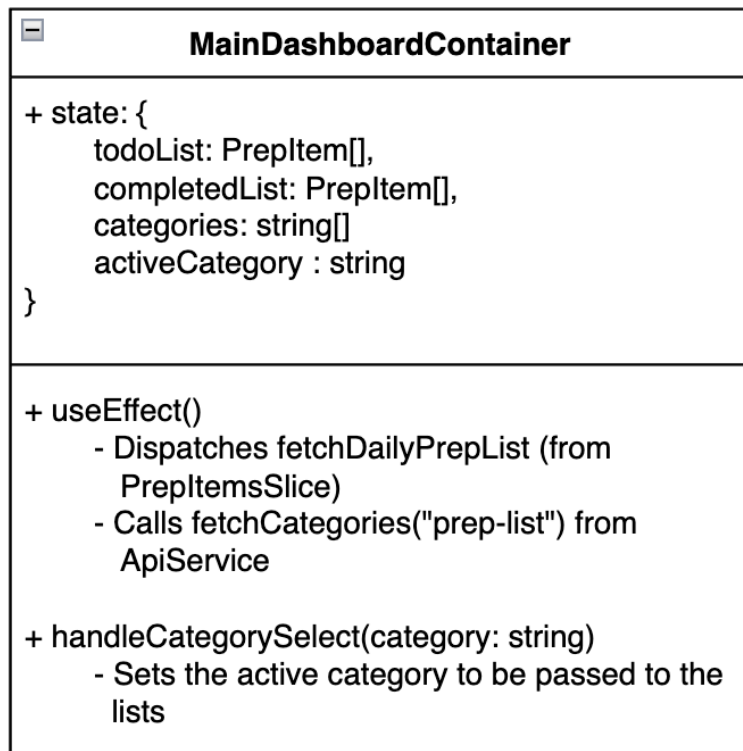
Prep Dash Nav:

Handles the updating of the prep dashboard search query and the active selection of screen to be displayed in the prep content area.



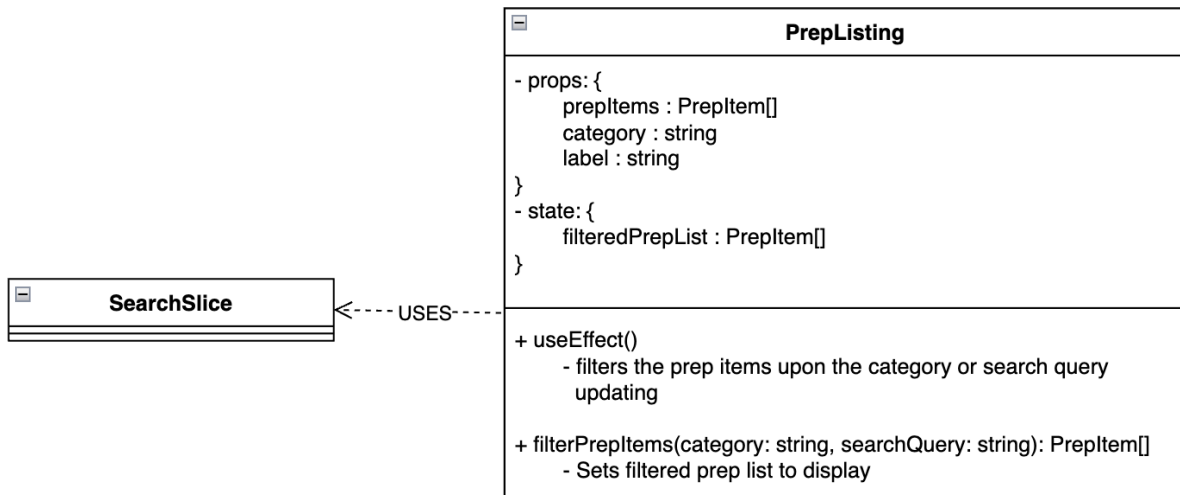
Main Dashboard Container:

Facilitates the population of all lists and category sliders, as well as triggers filtering by category.



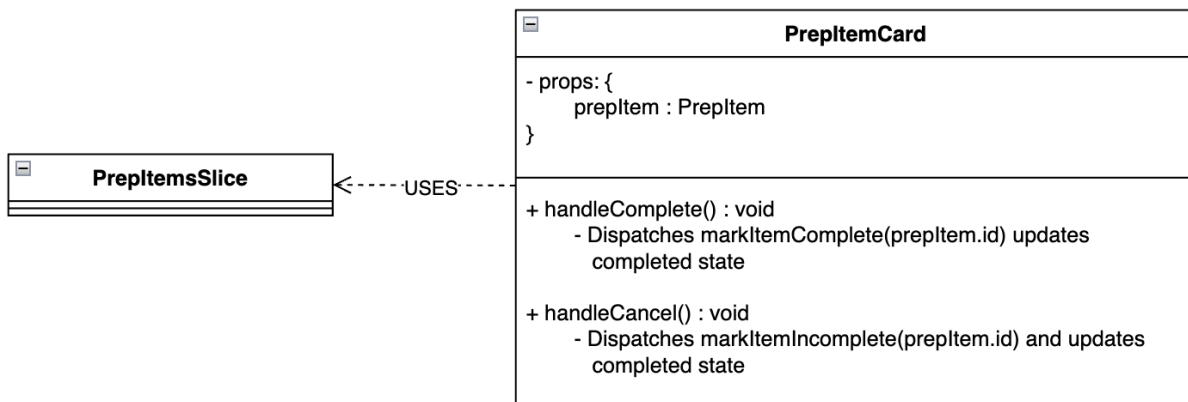
Prep Listing:

Upon updates to the category or (globally managed) search query, triggers a filter of the list.



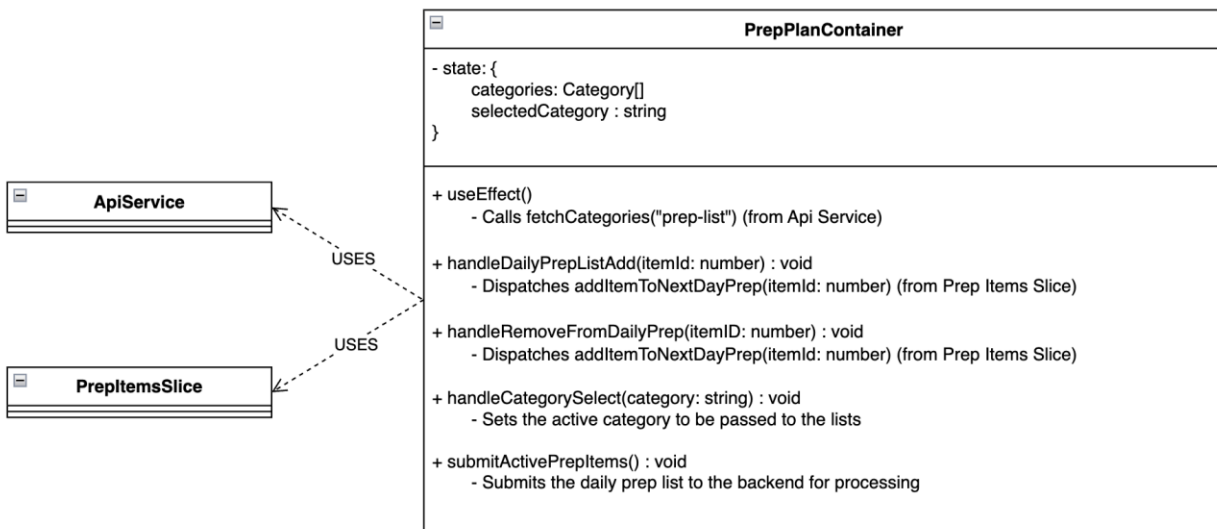
Prep Item Card:

Dispatches the functions that manage the prep completions.



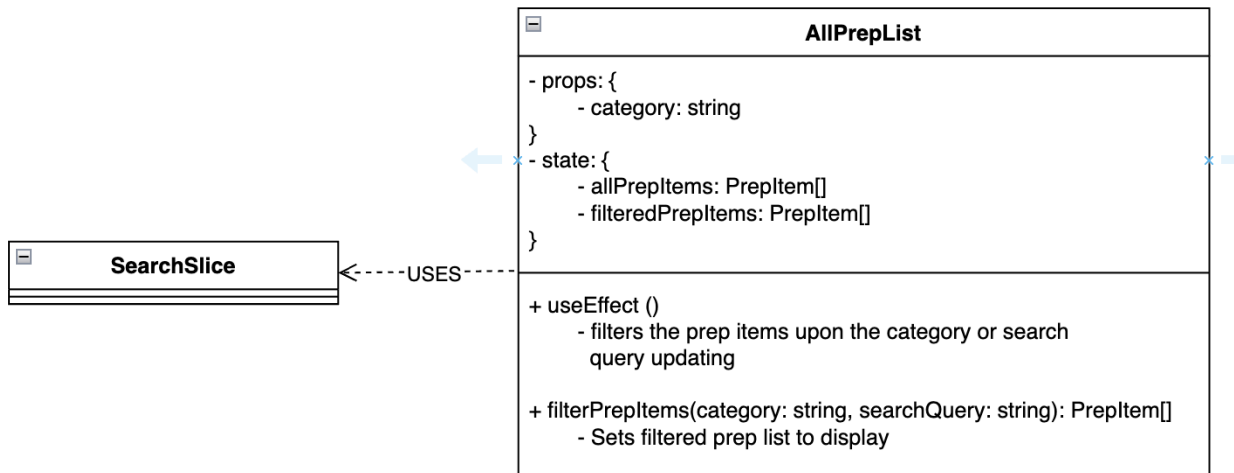
Prep Plan Container:

Fetches categories and category filtering, handles adding and deleting from the daily prep list, and finally submits the active prep items for the next day prep list.



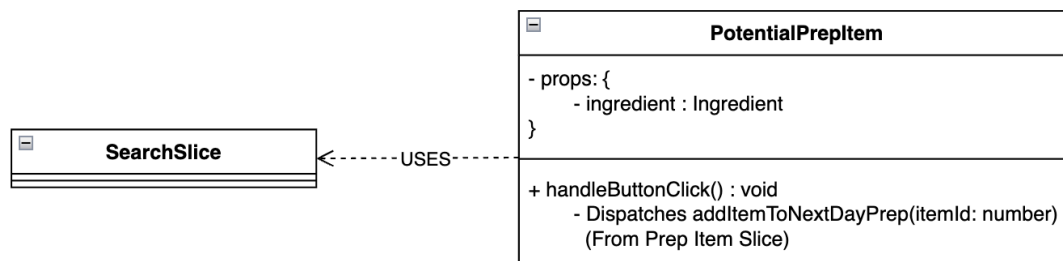
All Prep List:

Passed down ingredient names to all "Potential Prep Items" and filters the list upon a category being selected or the search query changing.



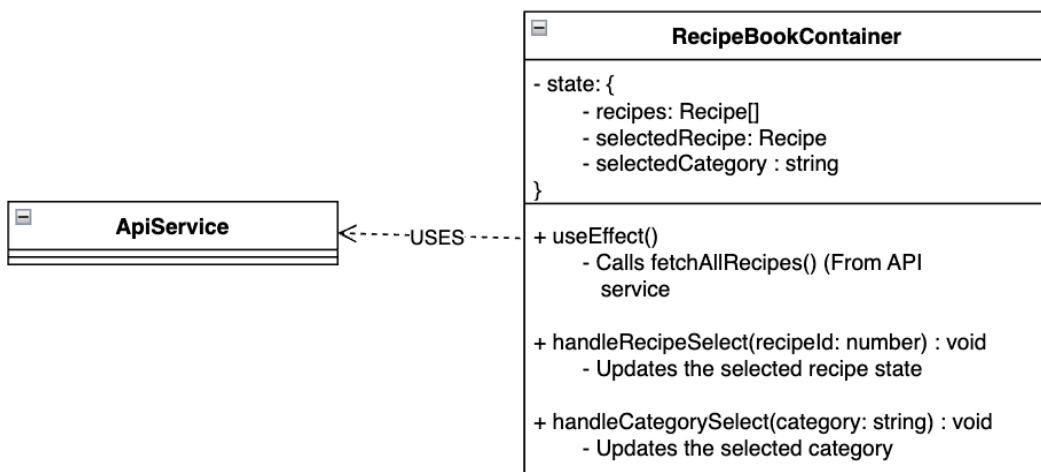
Potential Prep Item:

Handles the button click that adds an item to the next day's prep list.



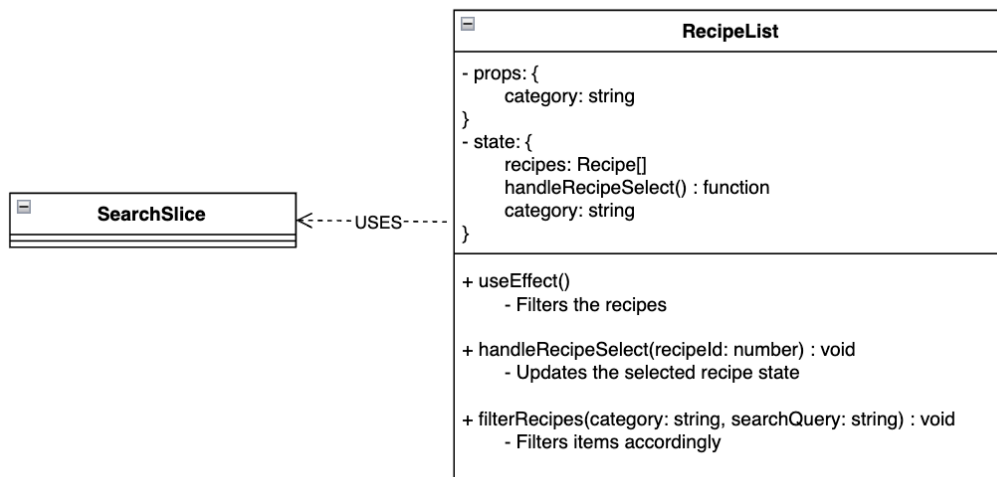
Recipe Book Container:

Fetches all recipes for a restaurant and handles category



Recipe List:

Displays all recipes, handles recipe click, and filters recipes according to category and search query.



Recipe Display:

Simply displays the recipe and does no handling of any extra state.

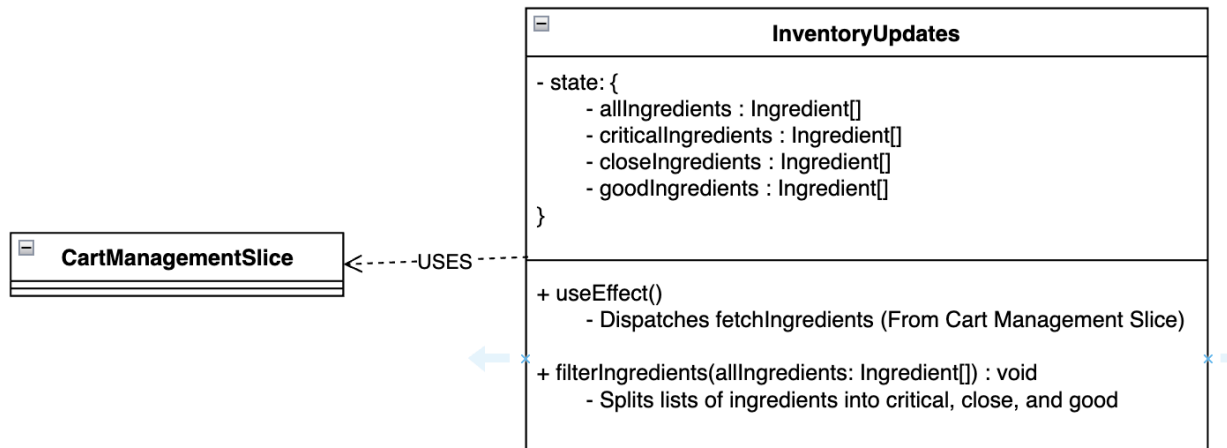
Manager Dash Nav:

Manager Dashboard Container:

It simply contains the three children's components that display the information.

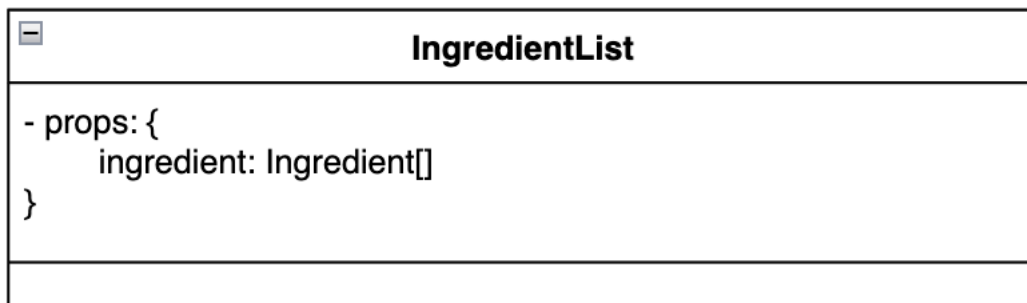
Inventory Updates:

Filters the list of ingredients into critical, close, and good lists to pass into the correct lists.



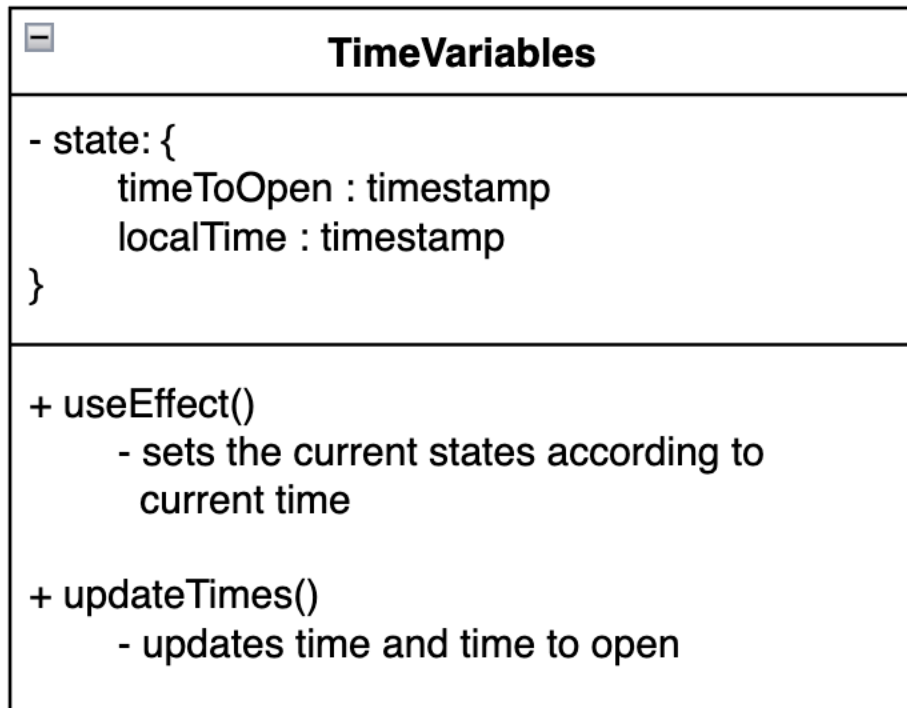
Ingredient List:

Displays the passed in list of ingredients.



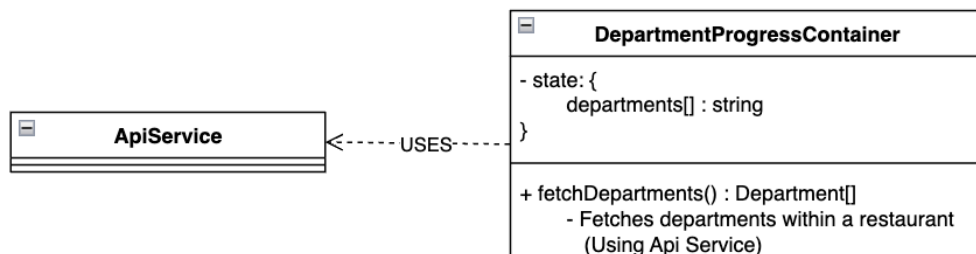
Time Variables:

Displays the opening time and local time, which are the same for all restaurants.



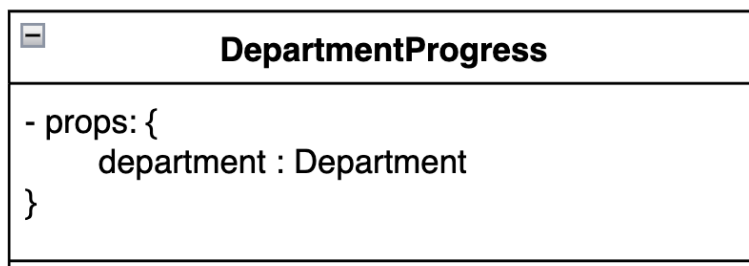
Department Progress Container:

Fetches all departments and their current progress for a current restaurant and passes them down to each progress bar.



Department Progress:

Simply displays department and updates department progress.



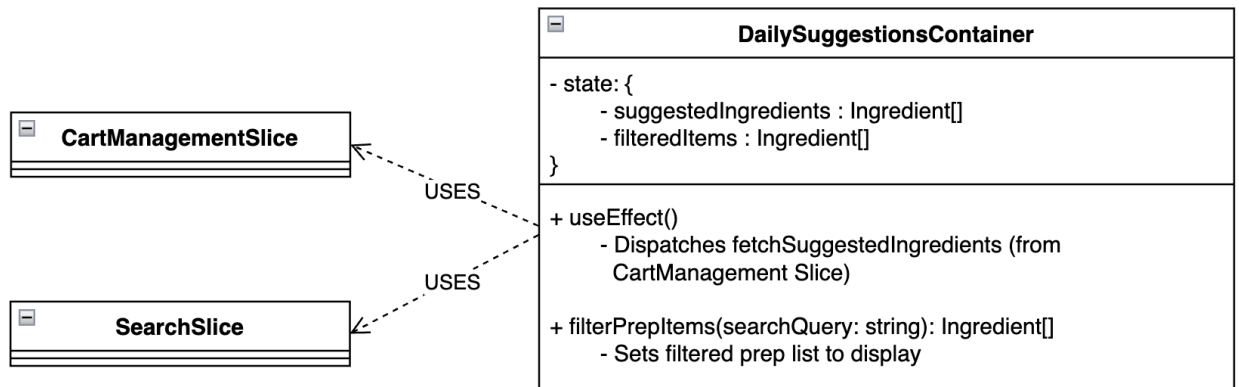
Market Page Container:

It is composed of numerous components and sets the search query globally.



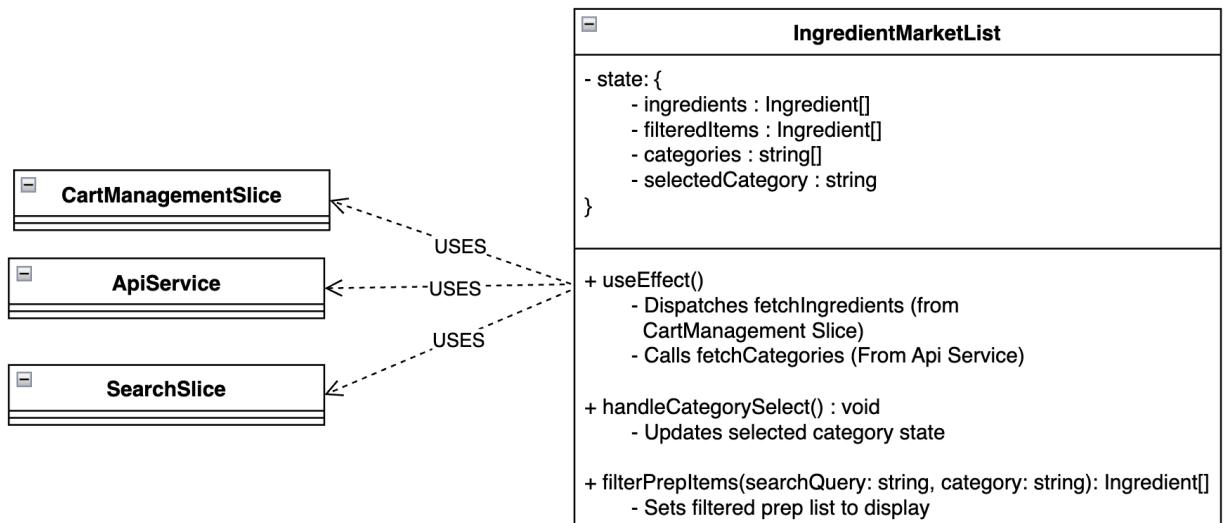
Daily Suggestions Container:

It is composed of suggestions determined by low inventory, and filters items based on the manager search query.



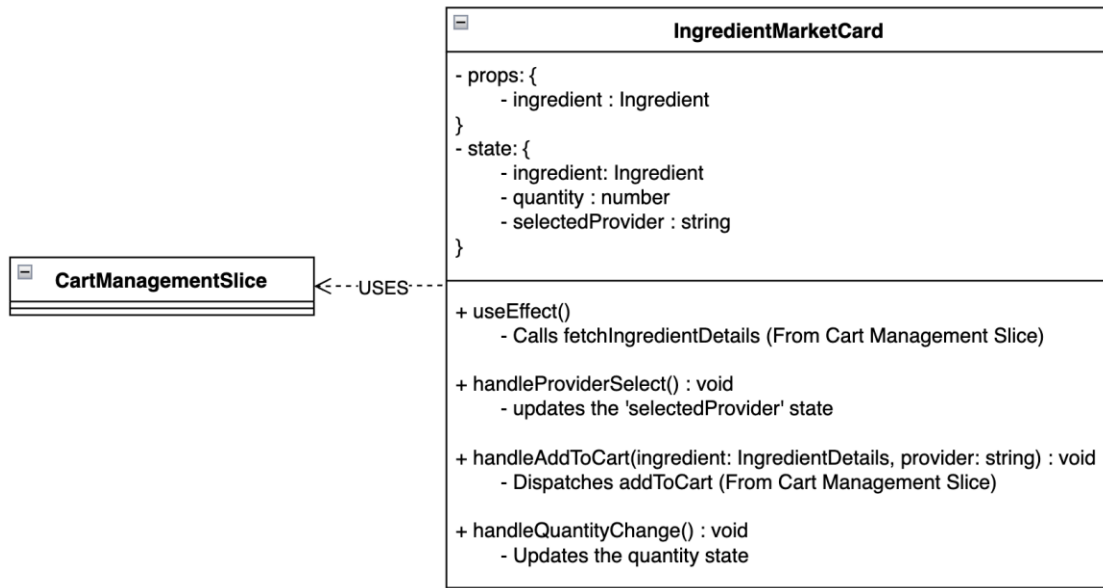
Ingredient Market List:

Displays all ingredients, handles category selection, and filters items by category and search query.



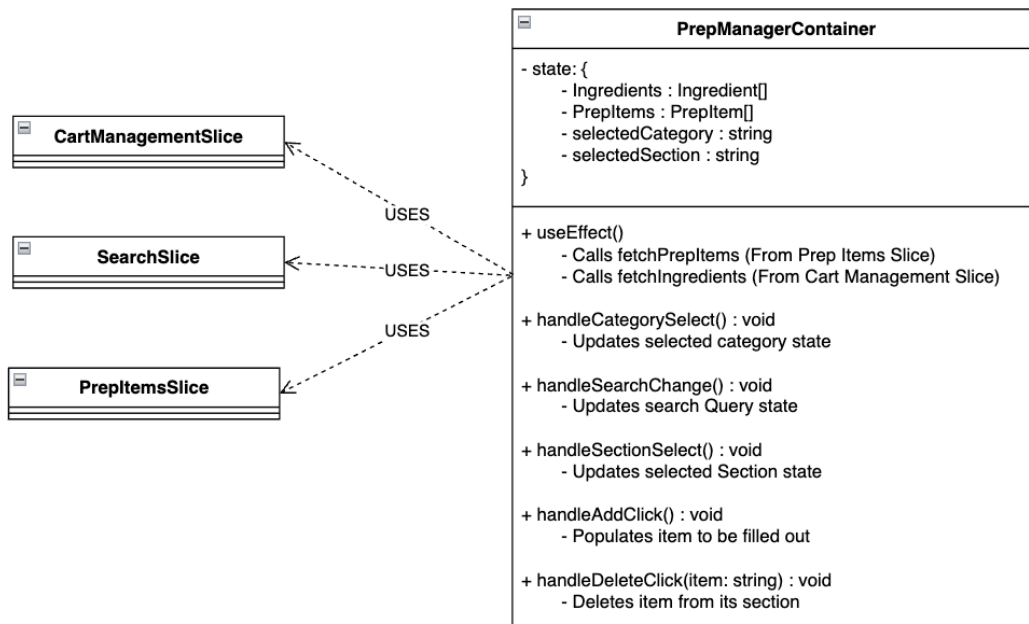
Ingredient Market Card:

Fetches all ingredients details, handles a provider select, handles an add to cart, and manages the quantity of an ingredient.



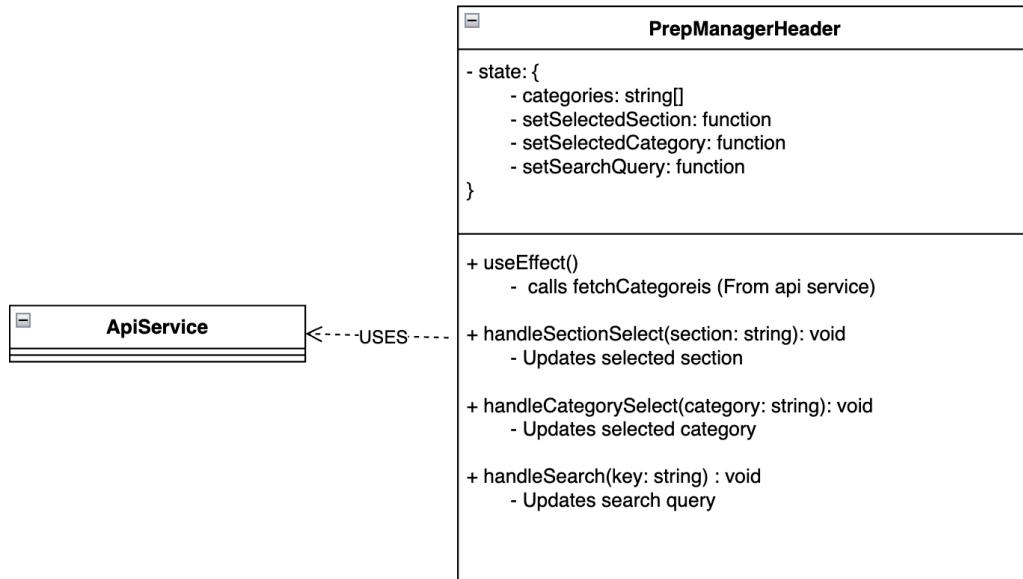
Prep Management Container:

Fetches all prep items and ingredients for a restaurant, updates state of all filtering options, handles add and delete behaviors.



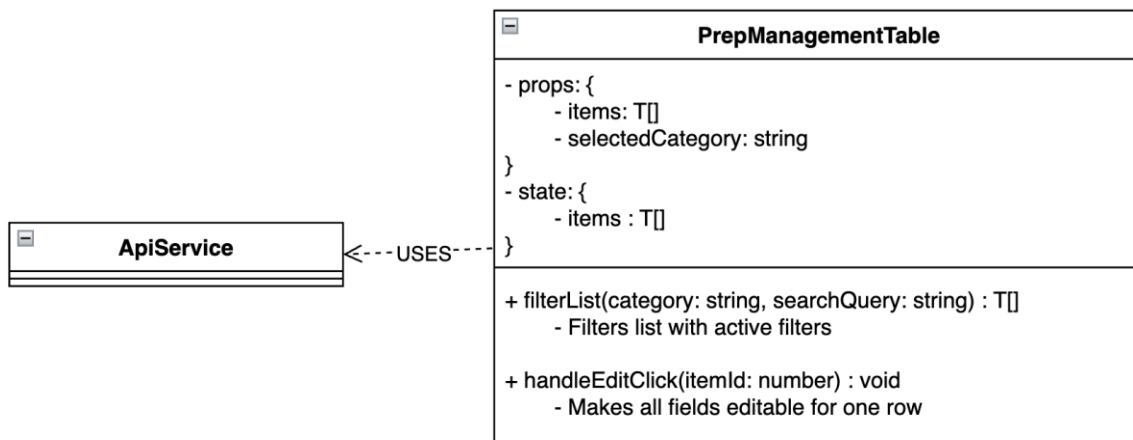
Prep Management Header:

Updates all state of filters and search queries.



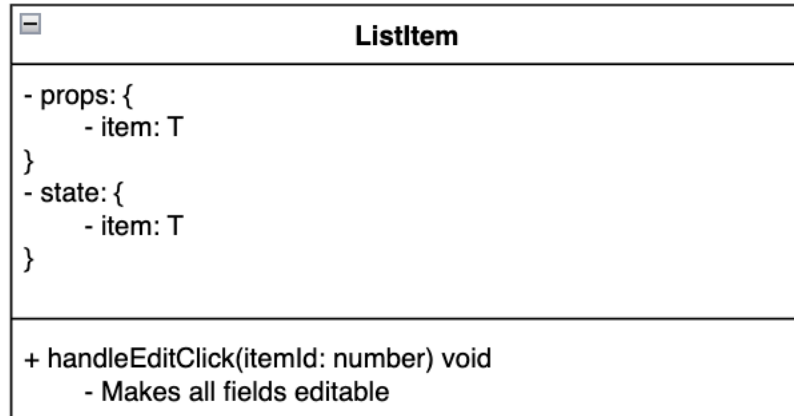
Prep Management Table:

Lists the items passed in and filters the list based on active filters.



List Item:

Lists the item and handles the edit button click.

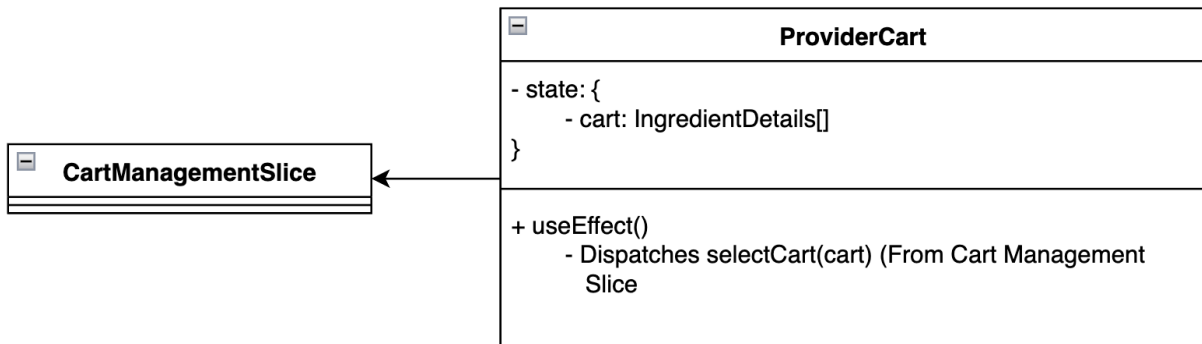


Cart Management Container:

It is composed of carts and the cart details components and holds no functionality.

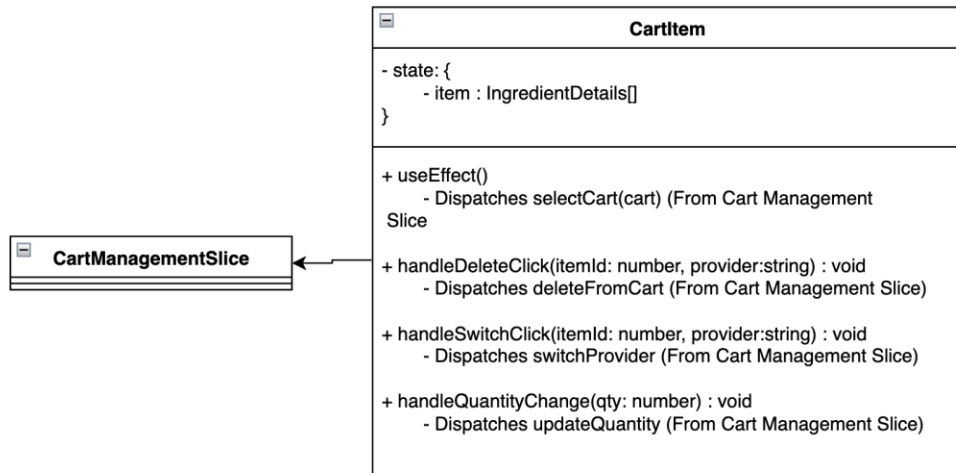
Provider Cart:

Fetch the items in the cart.



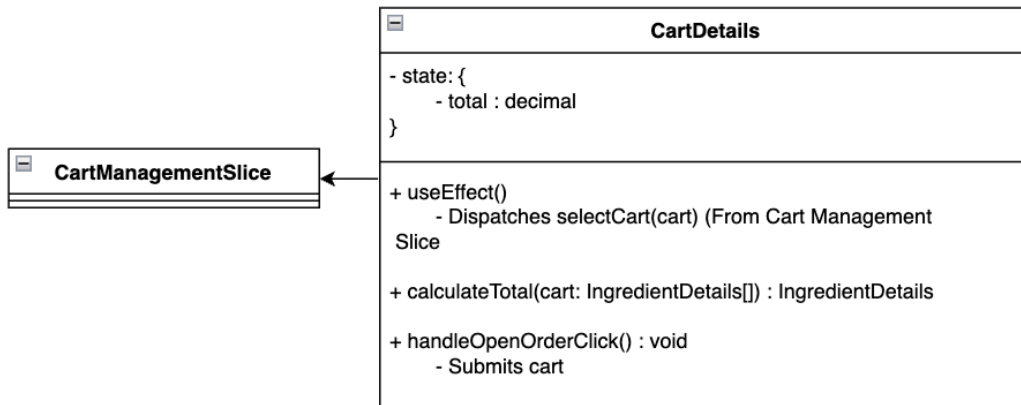
Cart Item:

Handles delete click, switch provider click, and handle quantity change.



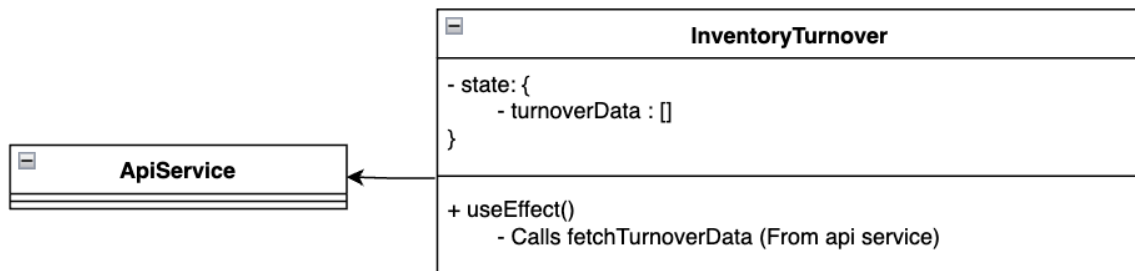
Cart Details:

Calculates the total of the specific cart and submits the data on button click.



Inventory Visualizer:

Links the tableau I Frame to backend data.

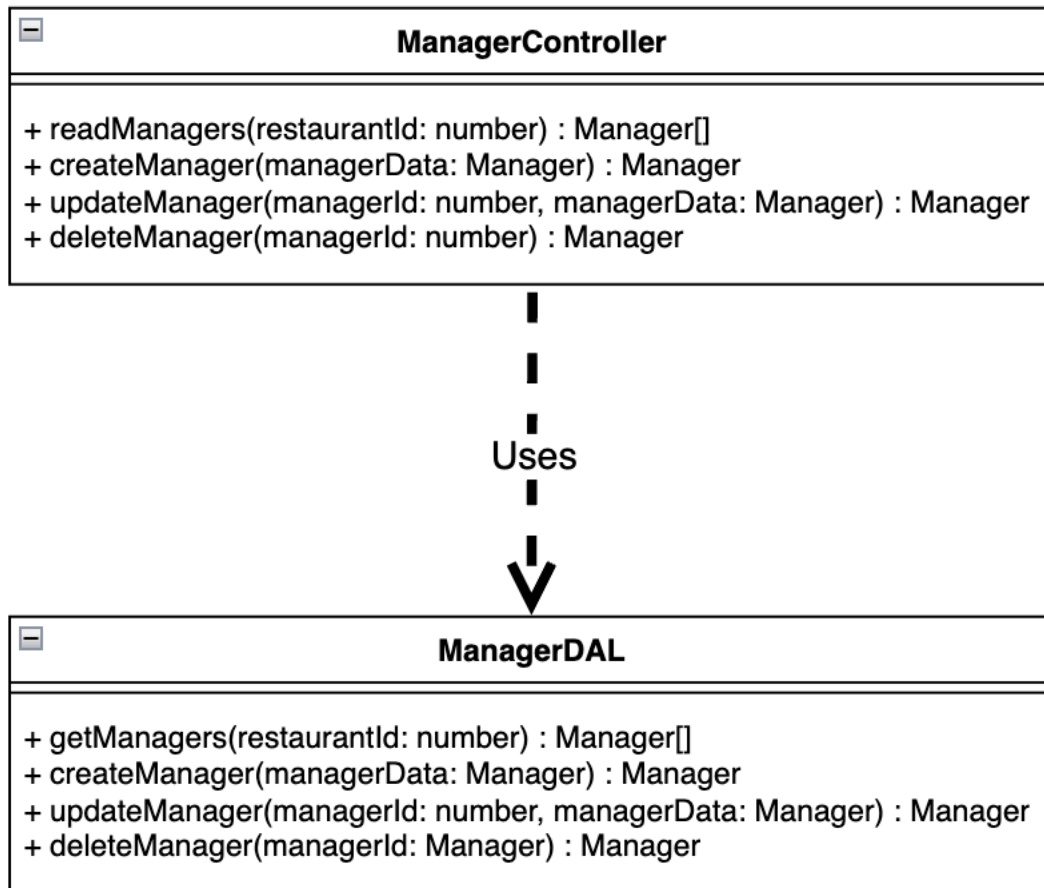


Backend Design:

The backend exposes resources for all data being displayed across the app. Here is a breakdown of each section of design specific to each resource:

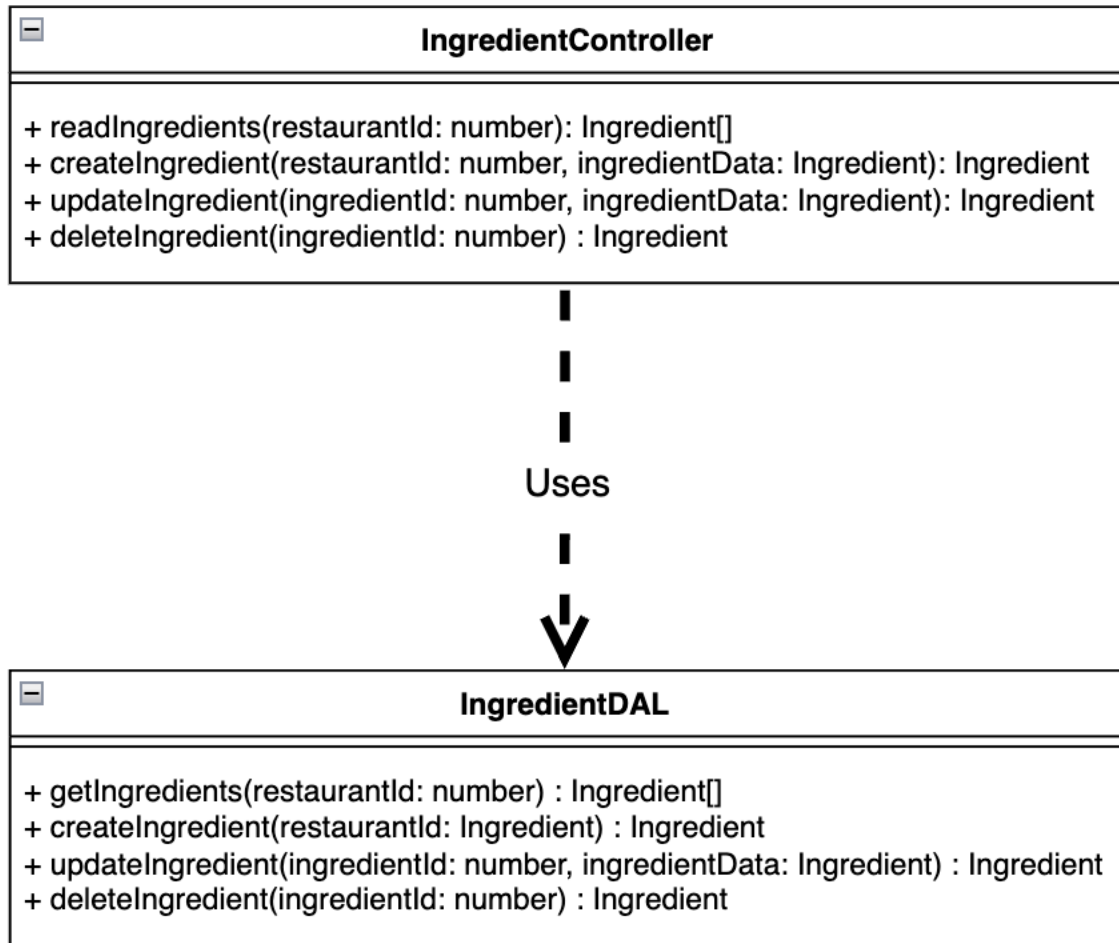
Managers Module:

Exposes 4 endpoints that access and modify the list of managers(admin) specific to one restaurant. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



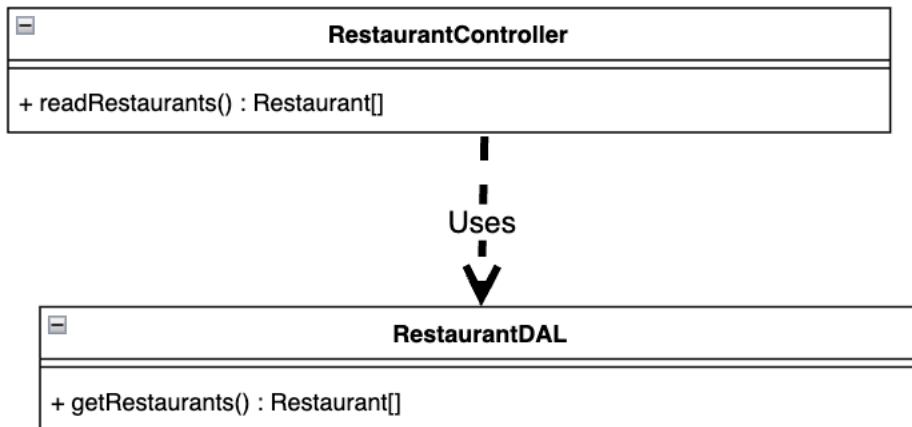
Ingredients Module:

Exposes 4 endpoints that access and modify the list of Ingredients specific to one restaurant. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



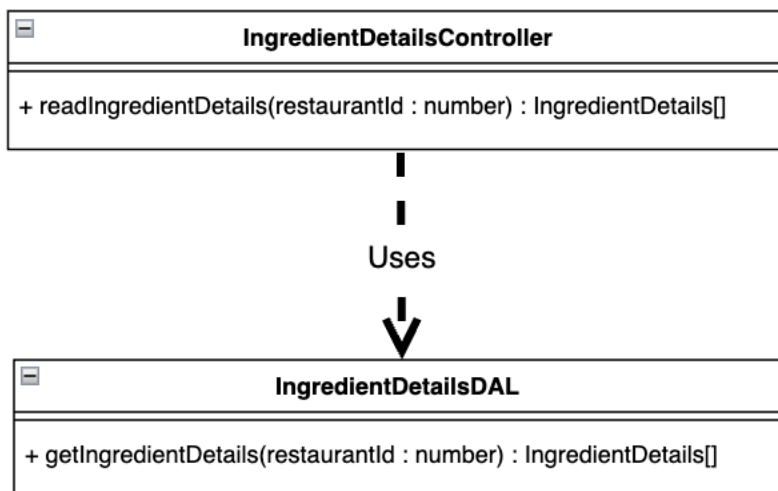
Restaurants Module:

Exposes 1 endpoint that accesses the list of Restaurants in the data store. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



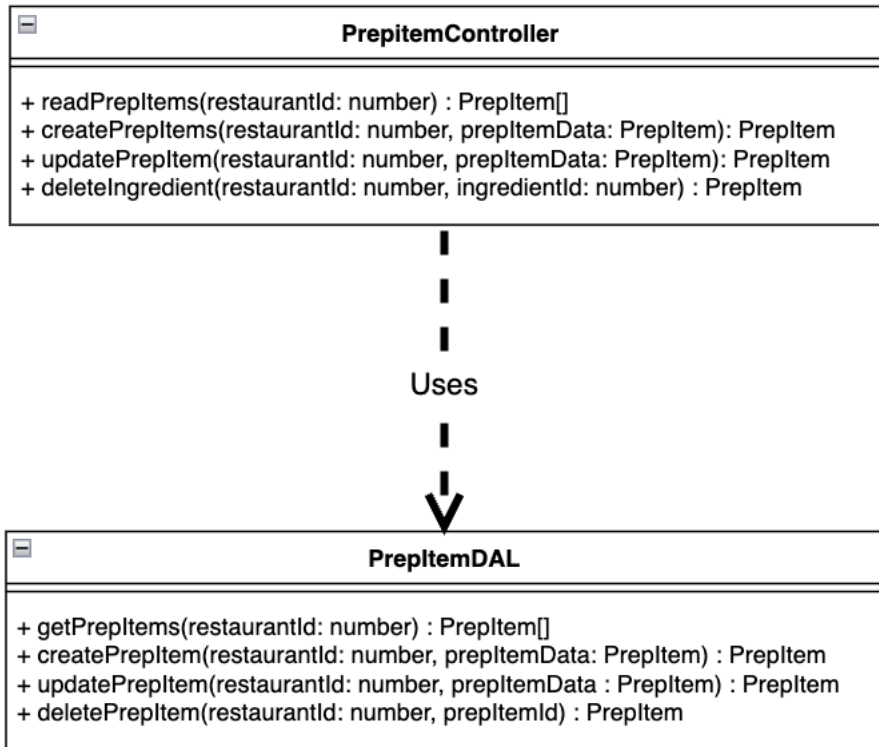
Ingredient Details Module:

Exposes 1 endpoint that accesses the list of Restaurants in the data store. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



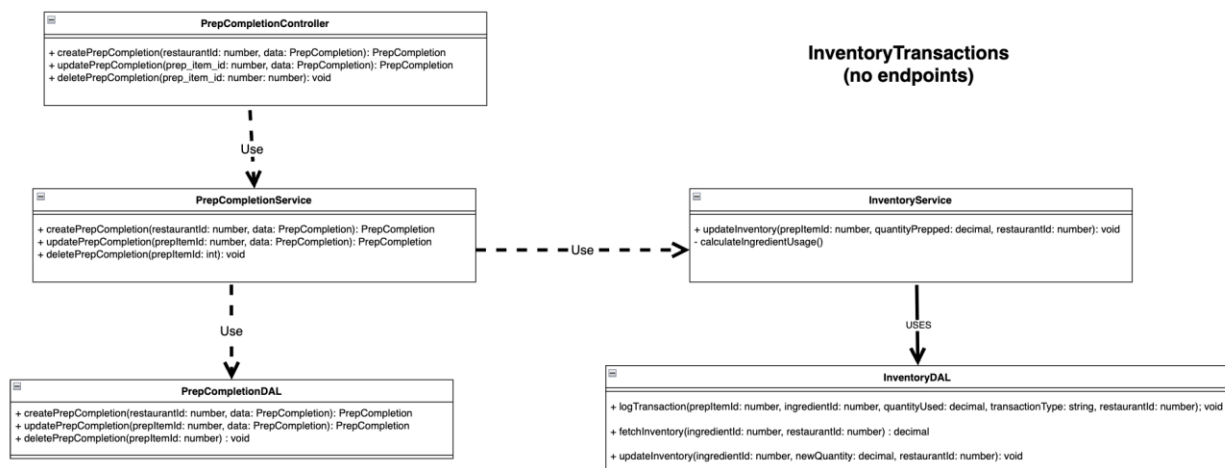
Prep Items Module:

Exposes 4 endpoints that access and modify the data store of Prep Items specific to one restaurant. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



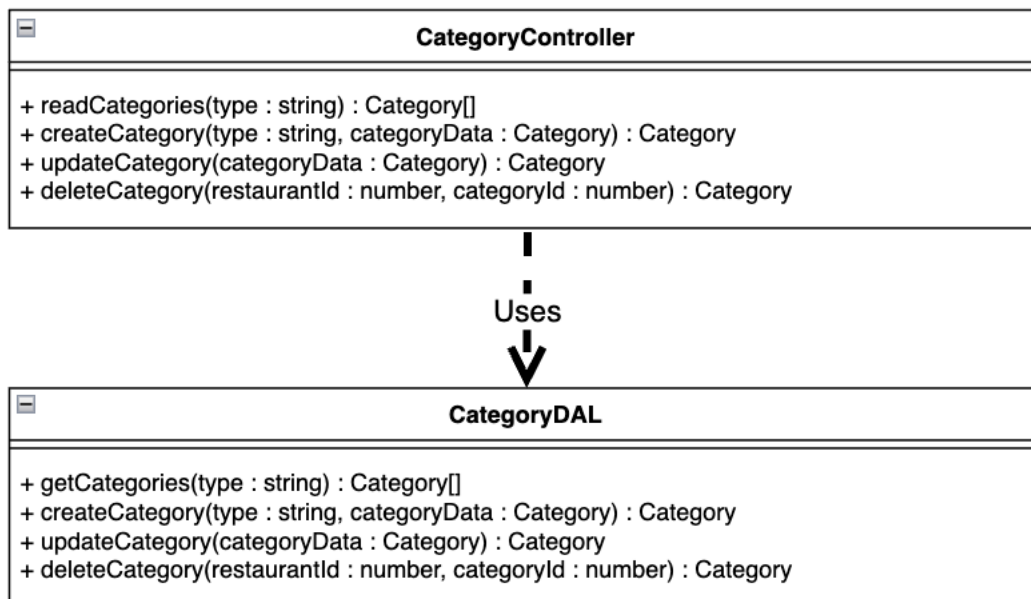
Prep Completion:

Exposes 4 endpoints that access and modify the data store of Prep Completions specific to one restaurant.



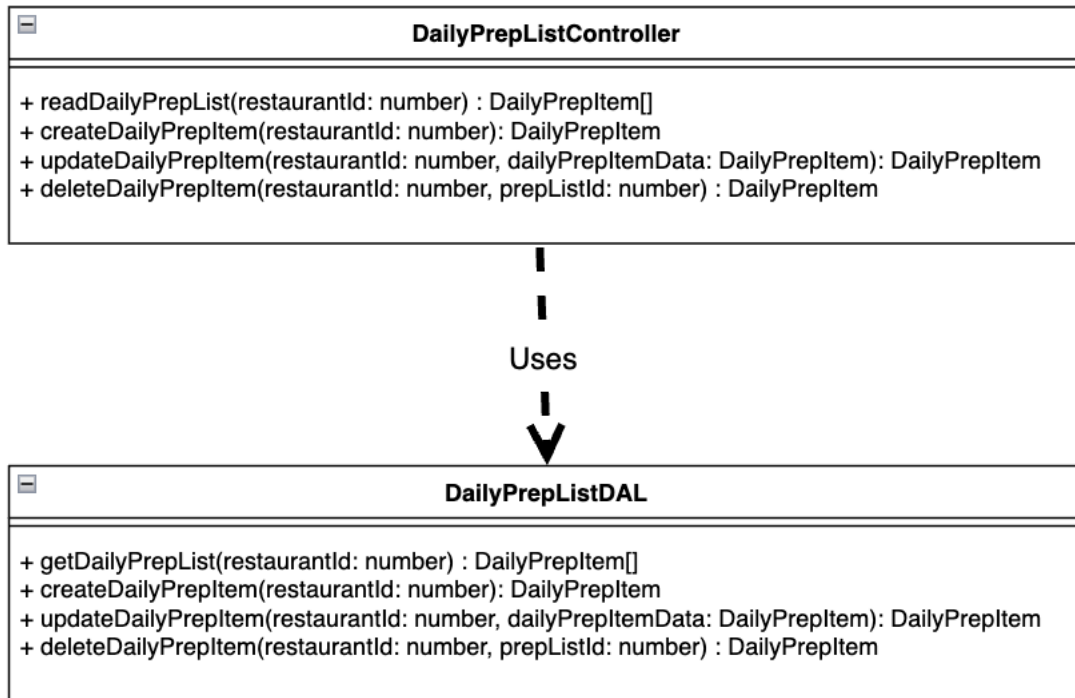
Categories:

Exposes 4 endpoint that access and modify the data store of Categories specific to one restaurant. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



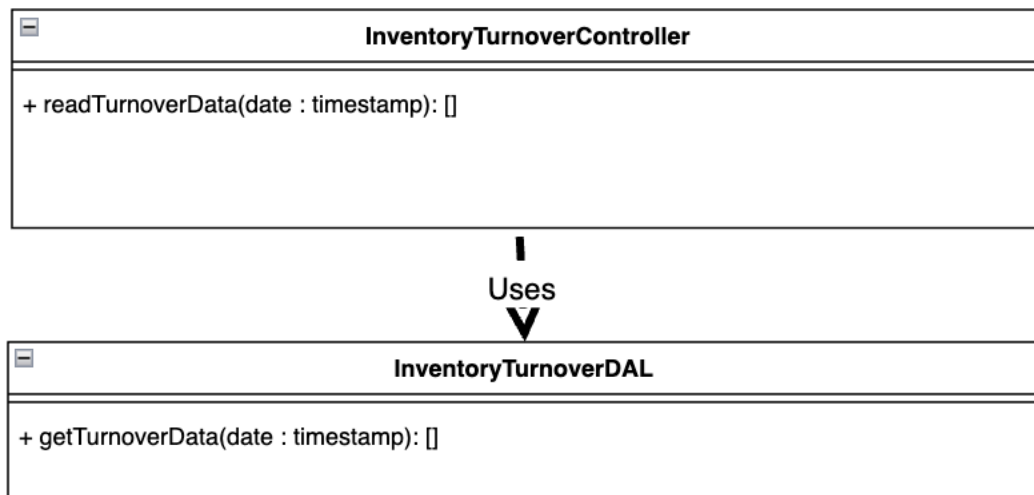
Daily Prep List:

Exposes 4 endpoints that access and modify the list of Ingredients specific to one restaurant. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



Inventory Turnover:

Exposes 1 endpoint that access and data specific to inventory turnover. This module only requires a controller and DAL, since there is no business logic along the path of data modification/store.



Service API Design:

I documented my service API using swagger. Before looking over the API documentation it's important to understand a few key points. There are numerous entities that function as a transactional log for the restaurant. By keeping these logs, it drives the insights and general

overview of restaurant operations that the app will offer. The API is a facade for nearly direct access for a lot of entities, making it RESTful, but there are also numerous service-oriented endpoints that require context and filters to create the return sets.

NFR's (Security Design, etc.):

My NFR I plan to support for Pi-Piper is **Data backup**. Because there is major emphasis upon the data driven components of my application, such as BI reports as detailed below, or daily suggestions defined by inventory management and ingredient pricing, ensuring that there are no holes in time-series data will ensure that my data is up to date and provides useful, accurate insights.

The backup redundancy I am aiming for is 24 hours. At worst there can be one day's worth of data loss. My decision of 24 hours is due to the fact that many decisions being made that go into data can be recalled in the case of a loss of a day of data. One day is about the biggest amount that would be possible to manually fill in, any more may induce some issues by user entry error.

The Recovery Point Objective (RPO) that I would like to aim for is sometime in the early morning, somewhere between 2am and 5am so that there is zero conflict with the restaurant operations. Sometimes there can be planning of prep or managerial duties that run into the early morning if there was a late closing the day before, but I figure there will never be employees or managers in at these hours no matter what the case. The key point that I want to hit with backing up data between these times is that the overhead of the application does not interfere with backing up, and flags thrown by a bad data backup can be signaled to management so it can be fixed first thing in the morning.

I plan to achieve this by purchasing a hard disk connected to my in-house hosting machine. There are numerous advantages that can be linked to utilizing a hard disk. First is accessibility, because hard disks generally utilize a wired connection, there is little overhead for backing up data since they can be accessed so quickly. It is also relatively safe disaster wise especially in my case where it is right next to the hosting machine, in case of disaster there is less chance that the data gets corrupted in the hard disk rather than the computer, which is more susceptible to water, or any other potentially harmful material.

The integrity of each backup should be checked upon backup completion, flagging any errors if data is incomplete or does not represent a full day's worth of data. Corrupted backups would be one of the most detrimental issues to plague my backup system, becoming unaware of poor backups would create problems if it had to be used to recover data and produce poor business insights.

Access control is an important aspect of this data backup system. Data backups and recoveries must only be initiated by authorized personnel. Failure to adhere by this principle could cause issues especially when trying to initiate a data recovery when not necessary, improperly overwriting data and likely creating errors. This will also ensure that when the data is backed up, it is out of the restaurant's hours of prep/service so that the performance overhead is focused on in-hours tools that may be used rather than backup services.

Operational Support Design:

The logging framework that will drive all of Pi Piper logging will be Winston. Logging will be completed when there is an attempt to access the REST facade in any fashion. If the request includes the authentication or a request for protected information, a User ID, response status, and response time will all be a part of the logging info. Otherwise, the response status, response time, and exact endpoint reached along with bodies and query parameters will be included in the format of logging that will be completed. Outside of the backend resources being reached, all major controller methods will log information for adequate debugging information. Outside of logging, monitoring will be completed by utilizing the free tier of Uptime Robot to receive notifications if there is ever stoppages in the application.

Appendix A – Technical Issue and Risk Log

1. Use the template to identify and monitor project issues and risks.

Issues and Risk Log								
Issue or Risk	Description	Project Impact	Action Plan/Resolution	Owner	Importance	Date Entered	Date to Review	Date Resolved
I/R	What is the issue or risk?	How will this impact scope, schedule, and cost?	How do you intend to deal with this issue?	Who manages this issue?				
R	Scraping script fails	This will severely affect the market screen by potentially using out-of-date costs that would change how a manager will arrange their carts.	To resolve this if there is an issue somewhere in the script this should be flagged and handled in the script by passing it back to the backend to be handled. The backend will then alert the frontend to show populate a modal that states the pricing is out of sync.	Scheduling module and data processing unit	HIGH	11/16	01/15	Unresolved
R	Wintel's capacity to manage future expansion of the project	The upfront cost of the Wintel and external hard drive is not cheap, but it would get extremely expensive if there had to be cost on top of this to host this across multiple sites. Time wise it would take far more planning and analysis and cost-wise would probably mean monthly hosting costs.	Upfront it's important I keep this in mind and try to limit the overhead of the application to begin with. It would make sense to be preemptive about this issue and begin to think about and document a potential cloud migration if it came to this.	Head Developer	MEDIUM	11/16	2/01	
R	Low/No Network Connectivity	In an environment where network connectivity may not be top of the line, it's important to account for this concern. If the network is going in and out without any safeguards, it can lead to a halt of business operations or using bad data to handle operations.	PWA addresses a lot of his ahead of time upon the first successful launch of the app caching the targeted data points. But there should be some extra caching and network checking done as an extra precaution. Thankful react query handles a lot of this caching workload beforehand.	Head Developer	MEDIUM	11/16	01/05	02/01
I	Maintaining logged-in managers across sessions.	Since there are some devices that everyone uses in the business there needs to be safeguards of the manager dashboard for safety of business insights and potential	There will always be a logout button in the manager navbar, but this is not the only way these screens will be protected. Recall that JWT will expire upon a set	Head Developer	MEDIUM	11/16	1/05	02/01

		cart manipulation. If these functionalities fell into the wrong hands, it could lead to the wrong people affecting prep list, or planning that may have a negative affect on the business.	amount of time as well so there will need to be a re login of the manager frequently.					
--	--	--	---	--	--	--	--	--

Appendix C – External Resources

GIT URL:	https://github.com/phollenback/Pi-Piper
----------	---